

✓ Classification du risque PFAS dans les eaux souterraines de Californie

Comparaison de 5 modeles — cible reglementaire EPA 2024 NPDWR

Travail de Master 2 — Recherche (Machine Learning geospatial applique aux PFAS)

Ce notebook compare **cinq approches** pour predire si un puits californien depasse les limites reglementaires PFAS de l'EPA 2024, **sans utiliser les concentrations PFAS** (protocole *reduced-feature* : on simule un puits non encore analyse en laboratoire).

#	Modele	Famille	Idee
1	HGT (Heterogeneous Graph Transformer)	Graphe / Deep Learning	coeur du travail de recherche — message-passing sur un graphe heterogene
2	Random Forest	Tabulaire (bagging)	baseline interpretable robuste
3	XGBoost	Tabulaire (boosting)	gradient boosting, etat de l'art tabulaire
4	Embedding Fusion	Hybride	embeddings HGT (PCA) + features tabulaires -> XGBoost
5	Stacking	Hybride (ensemble)	meta-classifieur sur (HGT + XGBoost + RandomForest)

Constat scientifique honnete : sur ce dataset, les modeles **tabulaires (XGBoost, RandomForest)** **egalent ou depassent le HGT**. L'interet du HGT reside dans sa **structure relationnelle** et il sert de **brique d'enrichissement** (embeddings, stacking). La derniere section quantifie la **generalisation reelle** en validation croisee spatiale.

Protocole et rigueur

- **Cible** : EPA 2024 NPDWR — depassement d'un MCL individuel **OU** Indice de Risque (HI) du melange > 1.
- **Anti-fuite** : exclusion des `*_ngL`, `*_detected`, `label_*`, et `gm_dataset_name` (proxy administratif).
- **Drop > 40 % de manquants** (regle imposee) : 31 colonnes retirees.
- **Split** stratifie 60/20/20, **identique pour les 5 modeles** (comparaison equitable).
- **Chaque modele** : optimisation **Optuna**, **courbe d'apprentissage**, **matrice de confusion**, **analyse SHAP**.
- **Section finale** : evaluation honnete en **CV spatiale par blocs (k=8)** + ablation geographique (desactivable).

Execution sur Google Colab

1. Activez un **runtime GPU** (`Execution > Modifier le type d'execution > T4 GPU`) pour acclereler le HGT.
2. Executez la cellule d'**installation**, puis celle de **chargement** du dataset (upload ou Drive).
3. Reglez `OPTUNA_MODE` et `RUN_SPATIAL_CV` dans la cellule de **configuration generale**.
4. La derniere cellule **zippe et telecharge** toutes les figures et tableaux de resultats.

✓ 0. Installation des librairies (Google Colab)

Installe PyTorch Geometric (compatible avec la version de torch presente sur Colab), XGBoost, LightGBM, Optuna, SHAP et imbalanced-learn. **A executer une seule fois** par session. En local (kernel deja equipe), la cellule se contente de verifier.

```
# --- Detection de l'environnement ---
import sys, importlib, subprocess
try:
    import google.colab # noqa: F401
    IN_COLAB = True
except Exception:
    IN_COLAB = False

def _pip(*pkgs):
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", *pkgs], check=False)

if IN_COLAB:
    print("Google Colab detecte — installation des dependances...")
    import torch
    tv = torch.__version__.split("+")[0]
    cu = ("cu" + torch.version.cuda.replace(".", "")) if torch.version.cuda else "cpu"
    print(f" torch={tv} | backend={cu}")
    _pip("torch_geometric")
    _pip("pyg_lib", "torch_scatter", "torch_sparse",
        "-f", f"https://data.pyg.org/whl/torch-{tv}+{cu}.html")
    _pip("xgboost", "lightgbm", "optuna", "shap", "imbalanced-learn", "seaborn", "pyarrow", "networkx")
    print("Installation terminee.")
else:
    print("Environnement local — verification des librairies (aucune installation).")
    for m in ["torch", "torch_geometric", "xgboost", "lightgbm", "optuna", "shap", "imblearn"]:
        ok = importlib.util.find_spec(m) is not None
        print(f" {m:16s}: {'OK' if ok else 'MANQUANT'}")
```

Google Colab detecte – installation des dependances...
 torch=2.11.0 | backend=cpu
 Installation terminee.

1. Imports

```
import warnings, json, pickle, time, os, zipfile
from pathlib import Path
# (re)definit IN_COLAB ici aussi, au cas ou la cellule d'installation aurait ete sautee
try:
    import google.colab # noqa: F401
    IN_COLAB = True
except Exception:
    IN_COLAB = False
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
warnings.filterwarnings("ignore")

from sklearn.model_selection import (train_test_split, StratifiedKFold,
    cross_validate, cross_val_predict, cross_val_score)
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import OrdinalEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.calibration import calibration_curve
from sklearn.metrics import (roc_auc_score, average_precision_score, f1_score,
    classification_report, ConfusionMatrixDisplay, confusion_matrix,
    roc_curve, precision_recall_curve, accuracy_score, recall_score,
    precision_score, balanced_accuracy_score)

import xgboost as xgb
import lightgbm as lgb
import optuna
optuna.logging.set_verbosity(optuna.logging.WARNING)

# Repli CPU : passez FORCE_CPU=True si le GPU Colab plante (CUDA device-side assert).
# A definir AVANT d'importer torch ; necessite ensuite un redemarrage du runtime.
FORCE_CPU = True
if FORCE_CPU:
    os.environ["CUDA_VISIBLE_DEVICES"] = ""
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.optim.lr_scheduler import ReduceLROnPlateau
try:
    from torch_geometric.data import HeteroData
    from torch_geometric.nn import HGTCnv, Linear as PyGLinear
    TORCH_GEOMETRIC_AVAILABLE = True
except Exception as e:
    TORCH_GEOMETRIC_AVAILABLE = False
    print("PyTorch Geometric indisponible :", e)

plt.rcParams.update({"figure.dpi": 110, "axes.spines.top": False,
    "axes.spines.right": False, "font.size": 11})
print("Imports OK | PyG disponible :", TORCH_GEOMETRIC_AVAILABLE)
```

Imports OK | PyG disponible : True

2. Configuration generale

Tous les reglages modifiables sont regroupes ici, chacun commente. Pour une execution rapide de bout en bout :

(OPTUNA_MODE="FAST" et RUN_SPATIAL_CV=False). Pour les resultats finaux du memoire : "FULL" (plusieurs heures, GPU recommande) + CV spatiale activee.

```
# =====
# CONFIGURATION GLOBALE – tout est reglable ici
# =====
RANDOM_STATE = 42 # graine globale (reproductibilite)

# --- Donnees ---
DATA_FILENAME = "CA-PFAS-ASGWS_v2.parquet" # nom du fichier dataset
MISSING_THRESHOLD = 0.40 # retrait de toute colonne avec > 40% de valeurs manquantes
```

```

DROP_LOCATION      = False # False = garde lat/lon/county... (requis par le graphe & la CV spatiale)
DEV_SUBSAMPLE      = None   # None = dataset complet ; int (ex 5000) pour un test rapide

# --- Split 60/20/20 ---
TEST_SIZE = 0.20      # part du test
VAL_SIZE  = 0.25      # part de validation prise sur le reste (=> 60/20/20)

# --- Desequilibre ---
USE_SMOTE = False     # dataset quasi-equilibre -> SMOTE optionnel (train-only)

# --- Optuna : FAST (rapide) ou FULL (resultats finaux) ---
OPTUNA_MODE = "FULL"
_TRIALS = {
    "FAST": {"rf": 8,  "xgb": 10, "hgt": 6,  "fusion": 8,  "stacking": 8},
    "FULL": {"rf": 40, "xgb": 60, "hgt": 15, "fusion": 20, "stacking": 20},
}[OPTUNA_MODE]
OPTUNA_TIMEOUT = {"FAST": 300, "FULL": 1800}[OPTUNA_MODE] # secondes max / study

# --- HGT ---
EPOCHS_HGT = {"FAST": 60, "FULL": 300}[OPTUNA_MODE]
N_GEO_CLUSTERS = 20      # noeuds geo_cluster du graphe heterogene

# --- Explicabilite ---
RUN_SHAP = True          # SHAP pour les modeles a arbres (rapide)
RUN_SHAP_HGT = True      # attribution par gradients du HGT (shap.GradientExplainer)
SHAP_SAMPLE = 1500       # nb d'echantillons test pour SHAP

# --- Section finale : evaluation honnete ---
RUN_SPATIAL_CV = False # CV spatiale par blocs k=8 (long). True pour le rendu final.
RUN_HGT_SPATIAL_CV = False # inclure le HGT dans la CV spatiale (~1-2 h CPU)
SPATIAL_CV_K = 8

# --- Reglementation EPA 2024 NPDWR ---
EPA2024_MCLS = {"PFOA_ngL": 4.0, "PFOS_ngL": 4.0, "PFNA_ngL": 10.0,
                "PFHxS_ngL": 10.0, "HFPO_DA_ngL": 10.0}
HI_REFS = {"PFNA_ngL": 10.0, "PFHxS_ngL": 10.0, "HFPO_DA_ngL": 10.0, "PFBS_ngL": 2000.0}

# --- Device & graines ---
# Seeding robuste : si le contexte CUDA est corrompu (device-side assert d'une
# execution precedente), on bascule proprement sur CPU au lieu de planter.
# -> en cas d'erreur CUDA persistante : Execution > Redemarrer la session.
np.random.seed(RANDOM_STATE)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
try:
    torch.manual_seed(RANDOM_STATE)
except Exception as _e:
    print("Avertissement CUDA :", _e)
    print("-> bascule sur CPU. Redemarrez le runtime (Execution > Redemarrer) pour reactiver le GPU.")
    device = torch.device("cpu")

# --- Sorties (figures + tableaux + modeles) ---
OUT_DIR = Path("outputs_notebook_ultime")
FIG_DIR = OUT_DIR / "figures"
RES_DIR = OUT_DIR / "results"
MODELS_DIR = OUT_DIR / "models"
for d in (FIG_DIR, RES_DIR, MODELS_DIR):
    d.mkdir(parents=True, exist_ok=True)

def savefig(fig, name):
    fig.savefig(FIG_DIR / name, dpi=150, bbox_inches="tight")

print(f"Mode Optuna : {OPTUNA_MODE} | trials={_TRIALS} | epochs HGT={EPOCHS_HGT}")
print(f"Device : {device} | torch {torch.__version__}")
print(f"Sorties -> {OUT_DIR.resolve()}")

Mode Optuna : FULL | trials={'rf': 40, 'xgb': 60, 'hgt': 15, 'fusion': 20, 'stacking': 20} | epochs HGT=300
Device : cpu | torch 2.11.0+cpu
Sorties -> /content/outputs_notebook_ultime

```

3. Chargement du dataset

Sur Colab : la cellule cherche le fichier localement, sinon propose un **upload** manuel (ou montez Google Drive en decomentant les lignes indiquees).

```

def locate_dataset(filename):
    candidates = [Path(filename), Path("data/processed")/filename,
                  Path("../data/processed")/filename, Path("/content")/filename,
                  Path("/content/drive/MyDrive")/filename]
    for c in candidates:
        if c.exists():

```

```

        return c
    if IN_COLAB:
        from google.colab import files
        print(f"Veuillez televerser '{filename}' :")
        up = files.upload()
        return Path(list(up.keys())[0])
        # --- Variante Google Drive (decommentez) :
        # from google.colab import drive; drive.mount('/content/drive')
        # return Path('/content/drive/MyDrive')/filename
    raise FileNotFoundError(f"{filename} introuvable – placez-le a cote du notebook.")

DATA_PATH = locate_dataset(DATA_FILENAME)
df = pd.read_parquet(DATA_PATH)
if DEV_SUBSAMPLE:
    df = df.sample(n=min(DEV_SUBSAMPLE, len(df)), random_state=RANDOM_STATE).reset_index(drop=True)
    print(f"[DEV] sous-echantillon : {len(df)} lignes")
print(f"Dataset : {df.shape[0]:,} lignes x {df.shape[1]} colonnes (source: {DATA_PATH})")
df.head(3)

```

Dataset : 46,338 lignes x 217 colonnes (source: CA-PFAS-ASGWS_v2.parquet)

	gm_well_id	collection_date	ADONA_ngL	F53B_major_ngL	F53B_minor_ngL	FTS_4_2_ngL	FTS_6_2_ngL	FTS_8_2_ngL	H
0	100001	2019-03-28	NaN	NaN	NaN	NaN	NaN	NaN	
1	100001	2020-04-28	1.0	1.0	1.0	NaN	NaN	NaN	
2	100002	2019-03-28	NaN	NaN	NaN	NaN	NaN	NaN	

3 rows x 217 columns

4. Construction de la cible EPA 2024 NPDWR

target = 1 si une concentration dépasse son MCL individuel **ou** si l'Indice de Risque du melange (HI) dépasse 1. Cible **recalculée** ici (le parquet **v2** n'a pas de colonne **target**). Concentrations manquantes traitées comme **0** (*non-detect = pas de dépassement*).

```

def compute_target_epa2024(d):
    exceeded = pd.Series(False, index=d.index)
    for col, mcl in EPA2024_MCLS.items():
        if col in d.columns:
            exceeded |= d[col].fillna(0) > mcl
    hi = sum(d[c].fillna(0) / r for c, r in HI_REFS.items() if c in d.columns)
    exceeded |= hi > 1.0
    return exceeded.astype(int)

```

```

df["target"] = compute_target_epa2024(df)
n_pos = int(df["target"].sum()); n_neg = len(df) - n_pos
print(f"Positifs (>= MCL ou HI>1) : {n_pos:,} ({100*n_pos/len(df):.2f}%)")
print(f"Negatifs (< MCL) : {n_neg:,} ({100*n_neg/len(df):.2f}%)")
print(f"Ratio negatif/positif : 1:{n_neg/n_pos:.2f} -> quasi-equilibre")

```

```

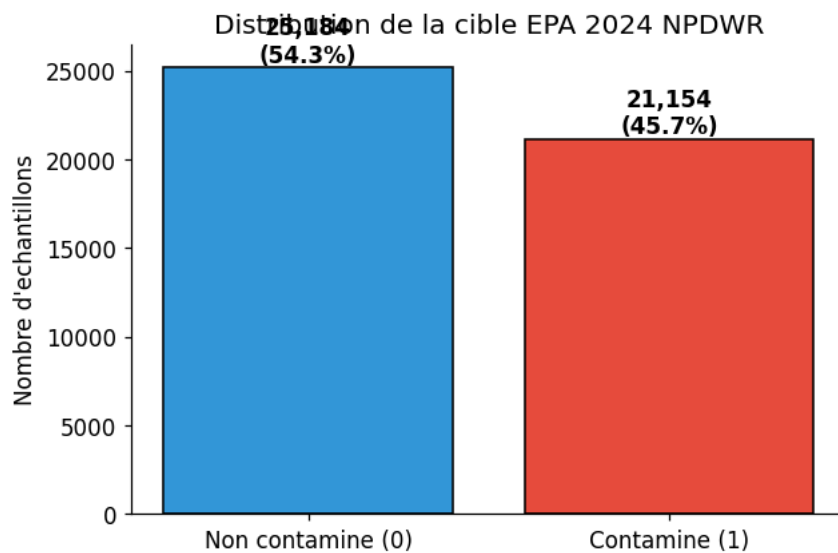
Positifs (>= MCL ou HI>1) : 21,154 (45.65%)
Negatifs (< MCL) : 25,184 (54.35%)
Ratio negatif/positif : 1:1.19 -> quasi-equilibre

```

```

# Figure 1 – distribution des classes
counts = df["target"].value_counts().sort_index()
fig, ax = plt.subplots(figsize=(6, 4))
bars = ax.bar(["Non contamined (0)", "Contamine (1)"], counts.values,
              color=["#3498db", "#e74c3c"], edgecolor="black")
for b, v in zip(bars, counts.values):
    ax.text(b.get_x()+b.get_width()/2, b.get_height(), f"{v:,}\n({100*v/len(df):.1f}%)",
            ha="center", va="bottom", fontweight="bold")
ax.set_ylabel("Nombre d'echantillons"); ax.set_title("Distribution de la cible EPA 2024 NPDWR")
plt.tight_layout(); savefig(fig, "fig01_class_distribution.png"); plt.show()

```



5. Prétraitement et ingénierie des features

Cette section a fait l'objet d'une **analyse dédiée** (taxonomie de la fuite, seuil de manquants, regroupement thématique). Le détail méthodologique est restitué ci-dessous, puis appliqué de manière **identique pour les 5 modèles**.

5.1 — Justification (niveau M2)

Pourquoi exclure la fuite (data leakage) — La cible est une **fonction déterministe** des concentrations PFAS. Toute variable qui réexprime cette information donnerait un score artificiellement parfait, sans valeur prédictive sur des forages non encore analysés. On retire donc quatre familles :

1. **_ngl** — les concentrations elles-mêmes (définissent la cible) ;
2. **_detected** — indicateurs de détection binaire (corr. jusqu'à 0,79 avec la cible) ;
3. **label** — labels de détection par molécule ;
4. **colonnes exactes** : `sum_pfas_ngl`, `pfas_class_assignment`, `gm_well_id`, `collection_date`, `target_sum_gt70`, `gldas_dist_km` (artefact de grille), et `gm_dataset_name` — *choix documenté* : c'est le nom du programme de surveillance, donc un **proxy administratif** fortement corrélé au protocole d'échantillonnage ; l'inclure reviendrait à apprendre « quel programme a échantillonné » plutôt que la physique du forage (fuite déguisée).

Le seuil de 40 % de manquants — Au-delà, l'imputation **invente la majorité** de la colonne (signal artificiel quasi-constant + risque de fuite via l'indicateur de manquant). **31 colonnes** dépassent 40 % : **17 sont des _ngl** (déjà éliminées comme fuite, retrait sans coût) et **14 sont de vraies features perdues** (co-contaminants rares, granulométrie fine du sol, géométrie de forage `well_depth_ft` / `screen_*` / `depth_eff_*`). Le signal hydrogéologique est partiellement récupéré par des proxys mieux renseignés et conservés (`depth_to_water_m`, `dist_nearest_gwl_km`, `hydr_grad_mag_permil`, `elevation_m`) avec leurs drapeaux `depth_missing` / `topo_missing` qui encodent l'absence **sans inventer** de valeur.

Redondance géographique — on retire `dwr_basin`, `sgma_basin_name`, `sgma_subbasin_name` (haute cardinalité administrative quasi-colinéaire avec lat/lon et les régions conservées).

Localisation conservée (DROP_LOCATION=False) — **risque de mémorisation spatiale** — on garde `latitude`, `longitude`, `county`, `regional_board`, `dwr_region`, `sgma_region_office` car (a) le graphe HGT en a besoin pour le voisinage spatial, (b) la CV spatiale en a besoin pour découper les blocs. **Risque assumé** : la contamination PFAS est fortement **auto-corrélée spatialement** (panaches autour de sites industriels / aéroports) ; une CV aléatoire **surestime** la performance. *On reporte donc en priorité les scores en CV spatiale* (section 13).

Prétraitement appliqué — numériques -> **imputation médiane** (robuste aux distributions asymétriques) ; catégorielles -> imputation « Unknown » + **OrdinalEncoder** (RF/XGBoost gèrent l'ordinal ; `soil_texture_class` a >100 modalités, one-hot exclu) ; pour le HGT -> **StandardScaler** en plus (un réseau de neurones exige des entrées centrées-réduites).

5.2 — Listes de features (vérifiées contre le fichier réel)

100 features retenues : **93 numériques** + **7 catégorielles**, partitionnées en **7 groupes thématiques** servant à construire les nœuds du graphe hétérogène HGT.

```
# — Sources de fuite (data leakage)
LEAK_SUFFIXES = ("_ngl",) # concentrations PFAS (fondent la cible)
LEAK_BOOL_SUFFIXES = ("_detected",) # indicateurs de détection
```

```

LEAK_PREFIXES      = ("label_",)      # labels de detection par molecule
LEAK_EXACT = {
    "sum_pfas_ngl", "pfas_class_assignment", "gm_well_id", "collection_date",
    "target", "target_sum_gt70", "gm_dataset_name", "gldas_dist_km",
}
DROP_REDUNDANT = {"dwr_basin", "sgma_basin_name", "sgma_subbasin_name"}

# — Features temporelles derivees de collection_date —————
if "collection_date" in df.columns:
    _dt = pd.to_datetime(df["collection_date"], errors="coerce")
    df["year"] = _dt.dt.year
    df["month"] = _dt.dt.month
    df["season"] = ((_dt.dt.month % 12) // 3 + 1).astype("float")

# — Drop > MISSING_THRESHOLD de manquants (calcule dynamiquement) —————
_miss = df.isna().mean()
HIGH_MISSING_DROP = set(_miss[_miss > MISSING_THRESHOLD].index)
_n_leak_hm = sum(1 for c in HIGH_MISSING_DROP if c.endswith("_ngl"))
print(f"Colonnes > {MISSING_THRESHOLD:.0%} manquants : {len(HIGH_MISSING_DROP)} "
      f"({_n_leak_hm} sont des _ngl/fuite, {len(HIGH_MISSING_DROP)-_n_leak_hm} vraies features)")

# — Localisation pure (retiree seulement si DROP_LOCATION=True) —————
LOCATION_FEATURES = {"latitude", "longitude", "county", "regional_board",
                   "dwr_region", "sgma_region_office"}
loc_drop = LOCATION_FEATURES if DROP_LOCATION else set()

# — Categorielles candidates —————
CATEGORICAL_CANDIDATES = ["county", "regional_board", "dwr_region", "gm_well_category",
                          "sgma_region_office", "nearest_geotracker_type", "soil_texture_class"]
cat_cols = [c for c in CATEGORICAL_CANDIDATES
            if c in df.columns and c not in loc_drop and c not in HIGH_MISSING_DROP]

# — Numeriques : tout le reste, hors fuite / manquants / redondances ———
def _is_leak(c):
    return (any(c.endswith(s) for s in LEAK_SUFFIXES)
            or any(c.endswith(s) for s in LEAK_BOOL_SUFFIXES)
            or any(c.startswith(p) for p in LEAK_PREFIXES)
            or c in LEAK_EXACT)

num_cols = [c for c in df.columns
            if df[c].dtype.kind in ("f", "i", "u")
            and not _is_leak(c) and c not in HIGH_MISSING_DROP
            and c not in DROP_REDUNDANT and c not in loc_drop and c not in cat_cols]

feature_names = num_cols + cat_cols
print(f"\nFeatures numeriques : {len(num_cols)} | categorielles : {len(cat_cols)} | TOTAL : {len(feature_names)}")

# — Verification anti-fuite —————
assert not any(_is_leak(c) for c in feature_names), "FUIITE detectee dans les features !"
assert not any(c in HIGH_MISSING_DROP for c in feature_names), "Colonne >40% manquants retenue !"
print("Verifications anti-fuite / manquants : OK")

```

Colonnes > 40% manquants : 31 (17 sont des _ngl/fuite, 14 vraies features)

Features numeriques : 96 | categorielles : 7 | TOTAL : 103
 Verifications anti-fuite / manquants : OK

```

# — Regroupement thematique des numeriques pour le graphe HGT —————
# Chaque feature numerique appartient a exactement un groupe ; un groupe "Other"
# capte d'eventuelles colonnes nouvelles non prevues (robustesse si le schema evolue).
_GROUPS_SPEC = {
    "Geospatial":      ["latitude", "longitude"],
    "Facility_Proximity": ["dist_geotracker_km", "n_geotracker_within_1km",
                        "n_geotracker_within_3km", "n_geotracker_within_10km",
                        "n_geotracker_within_50km"],
    "Water_Quality_cocontam": [c for c in num_cols if c.startswith("cocontam_")],
    "Soil_SSURGO":      [c for c in num_cols if c.startswith("soil_")
                        and not c.startswith("soil_moi") and c != "soil_moisture_total_mm"],
    "Air_AQS":          [c for c in num_cols if c.startswith("aqs_")],
    "Environmental_GLDAS_hydro": ["rainfall_mm_month", "et_mm_month", "runoff_mm",
                                "soil_moi_0_10_kg_m2", "soil_moi_10_40_kg_m2",
                                "soil_moi_40_100_kg_m2", "soil_moi_100_200_kg_m2",
                                "root_zone_moist_kg_m2", "temp_c", "snowpack_mm",
                                "soil_moisture_total_mm"],
    "Hydrogeology":    ["depth_to_water_m", "depth_to_water_loglp", "dtw_far",
                        "depth_missing", "elevation_m", "topo_missing",
                        "dev_intensity", "lc_developed", "hydr_grad_mag_permil",
                        "flow_dir_sin", "flow_dir_cos", "dist_nearest_gwl_km"],
    "Temporal":        [c for c in ["year", "month", "season"] if c in num_cols],
}
FEATURE_GROUPS, _assigned = {}, set()
for g, feats in _GROUPS_SPEC.items():

```

```

keep = [f for f in feats if f in num_cols and f not in _assigned]
if keep:
    FEATURE_GROUPS[g] = keep; _assigned.update(keep)
_other = [c for c in num_cols if c not in _assigned]
if _other:
    FEATURE_GROUPS["Other"] = _other

print("Groupes thematiques (noeuds du graphe HGT) :")
for g, feats in FEATURE_GROUPS.items():
    print(f" {g:26s}: {len(feats):2d} features")
print(f" {'TOTAL numeriques':26s}: {sum(len(v) for v in FEATURE_GROUPS.values())}")

```

```

Groupes thematiques (noeuds du graphe HGT) :
Geospatial           : 2 features
Facility_Proximity    : 5 features
Water_Quality_cocontam : 39 features
Soil_SSURGO           : 16 features
Air_AQS               : 8 features
Environmental_GLDAS_hydro : 11 features
Hydrogeology          : 12 features
Temporal              : 3 features
TOTAL numeriques      : 96

```

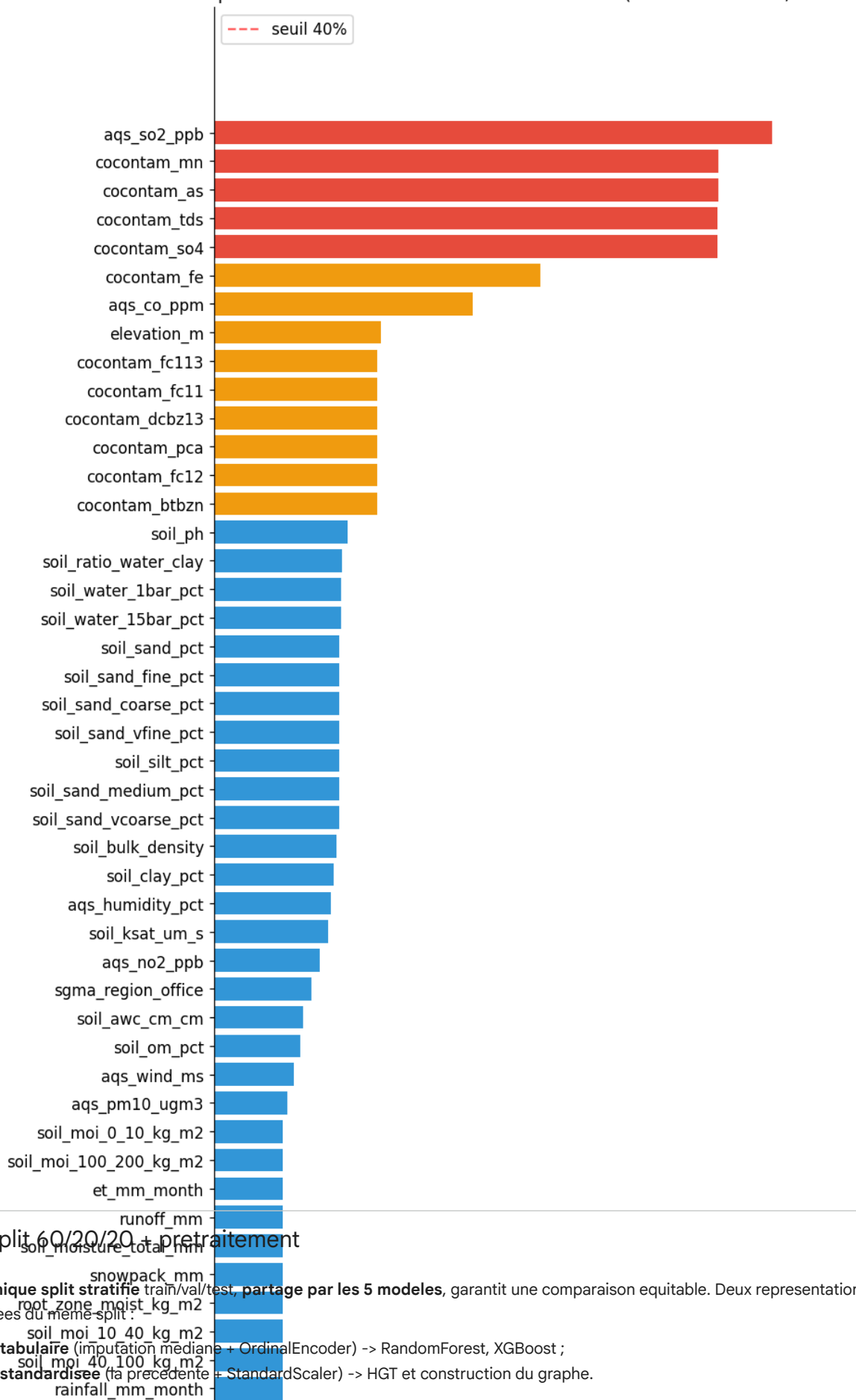
5.3 — Profil des valeurs manquantes (features retenues)

```

# Figure 2 – taux de manquants des features retenues (avant imputation)
miss_feat = df[feature_names].isna().mean().sort_values(ascending=False)
miss_top = miss_feat[miss_feat > 0]
fig, ax = plt.subplots(figsize=(9, max(3, len(miss_top)*0.32)))
if len(miss_top):
    cols = ["#e74c3c" if v > 0.3 else "#f39c12" if v > 0.1 else "#3498db" for v in miss_top]
    ax.barh(miss_top.index[::-1], miss_top.values[::-1], color=cols[::-1])
    ax.axvline(MISSING_THRESHOLD, color="red", ls="--", alpha=.6, label=f"seuil {MISSING_THRESHOLD:.0%}")
    ax.set_xlabel("Taux de valeurs manquantes"); ax.legend()
    ax.set_title(f"Manquants residuels des {len(feature_names)} features retenues "
                 f"({len(miss_top)} ont des trous, toutes < {MISSING_THRESHOLD:.0%})")
else:
    ax.text(.5, .5, "Aucune valeur manquante", ha="center")
plt.tight_layout(); savefig(fig, "fig02_missingness.png"); plt.show()

```


Manquants residuels des 103 features retenues (80 ont des trous, toutes < 40%)



6. Split 60/20/20 + pretraitement

Un **unique split stratifié** train/val/test, **partage par les 5 modeles**, garantit une comparaison equitable. Deux representations sont derivees du meme split :

- **tabulaire** (imputation mediane + OrdinalEncoder) -> RandomForest, XGBoost ;
- **standardisee** (la precedente + StandardScaler) -> HGT et construction du graphe.

```
X = df[feature_names].copy()
y = df["target"].values

# Split 60/20/20 (val pris sur le reste apres avoir mis le test de cote)
X_tmp, X_test, y_tmp, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, stratify=y, random_state=RANDOM_STATE)
```

```

X_train, X_val, y_train, y_val = train_test_split(
    X_tmp, y_tmp, test_size=VAL_SIZE, stratify=y_tmp, random_state=RANDOM_STATE)
print(f"Train: {len(y_train):,} ({100*y_train.mean():.1f}% +) | "
      f"Val: {len(y_val):,} ({100*y_val.mean():.1f}% +) | "
      f"Test: {len(y_test):,} ({100*y_test.mean():.1f}% +)")

# — Préprocesseur tabulaire (arbres) : impute median + ordinal encode —
numeric_pipe = Pipeline([("imputer", SimpleImputer(strategy="median"))])
categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="constant", fill_value="Unknown")),
    ("encoder", OrdinalEncoder(handle_unknown="use_encoded_value",
                               unknown_value=-1, encoded_missing_value=-1)),
])
preprocessor = ColumnTransformer([
    ("num", numeric_pipe, num_cols),
    ("cat", categorical_pipe, cat_cols),
], remainder="drop")

X_train_proc = preprocessor.fit_transform(X_train)
X_val_proc = preprocessor.transform(X_val)
X_test_proc = preprocessor.transform(X_test)
proc_cols = num_cols + cat_cols # ordre des colonnes en sortie du ColumnTransformer

# — Representation standardisee (HGT) —
scaler = StandardScaler().fit(X_train_proc)
X_train_sc = scaler.transform(X_train_proc)
X_val_sc = scaler.transform(X_val_proc)
X_test_sc = scaler.transform(X_test_proc)

# — SMOTE optionnel (train uniquement) —
# Important : on resample la matrice tabulaire UNE SEULE FOIS puis on re-derive
# la version standardisee, pour garder l'alignement des lignes entre les deux
# representations (nécessaire pour le stacking & la fusion).
if USE_SMOTE:
    from imblearn.over_sampling import SMOTE
    sm = SMOTE(random_state=RANDOM_STATE)
    X_train_proc, y_train_bal = sm.fit_resample(X_train_proc, y_train)
    y_train_bal = np.asarray(y_train_bal)
    X_train_sc = scaler.transform(X_train_proc)
    print(f"SMOTE applique : train -> {len(y_train_bal)} lignes")
else:
    y_train_bal = y_train
print(f"X_train_proc {X_train_proc.shape} | X_train_sc {X_train_sc.shape}")

```

```

Train: 27,802 (45.7% +) | Val: 9,268 (45.7% +) | Test: 9,268 (45.7% +)
X_train_proc (27802, 191) | X_train_sc (27802, 103)

```

7. Fonctions utilitaires (metriques, courbes, SHAP)

Le **rappel (recall)** de la classe positive est la metrique prioritaire en contexte sanitaire : un faux negatif = puits contamine non signale.

```

RESULTS = {} # collecte des metriques de chaque modele pour la comparaison finale

def evaluate(name, y_true, y_pred, y_proba, store=True, show=True):
    m = {"name": name,
         "roc_auc": roc_auc_score(y_true, y_proba),
         "avg_precision": average_precision_score(y_true, y_proba),
         "accuracy": accuracy_score(y_true, y_pred),
         "balanced_accuracy": balanced_accuracy_score(y_true, y_pred),
         "precision": precision_score(y_true, y_pred, zero_division=0),
         "recall": recall_score(y_true, y_pred),
         "f1": f1_score(y_true, y_pred)}
    if show:
        print(f"{'-'*52}\n {name}\n{'-'*52}")
        for k in ["roc_auc", "avg_precision", "recall", "precision", "f1", "balanced_accuracy", "accuracy"]:
            print(f" {k:20s}: {m[k]:.4f}")
    if store:
        RESULTS[name] = {"metrics": m, "y_pred": np.asarray(y_pred), "y_proba": np.asarray(y_proba)}
    return m

def plot_confusion(name, y_true, y_pred, fname):
    fig, ax = plt.subplots(figsize=(4.3, 4))
    ConfusionMatrixDisplay.from_predictions(
        y_true, y_pred, display_labels=["< MCL (0)", ">= MCL (1)"],
        cmap="Blues", ax=ax, colorbar=False)
    ax.set_title(f"Matrice de confusion - {name}")
    plt.tight_layout(); savefig(fig, fname); plt.show()

def optuna_history_fig(study, title, fname, color="steelblue"):
    fig, axes = plt.subplots(1, 2, figsize=(13, 4))

```

```

vals = [t.value for t in study.trials if t.value is not None]
best = [max(vals[i+1]) for i in range(len(vals))]
axes[0].plot(vals, "o", alpha=.4, color=color, ms=5, label="Trial")
axes[0].plot(best, "-", color="black", lw=2, label="Meilleur cumule")
axes[0].set(xlabel="Trial", ylabel="Score objectif", title=f"{title} – historique Optuna")
axes[0].legend(); axes[0].grid(alpha=.3)
try:
    imp = optuna.importance.get_param_importances(study)
    axes[1].barh(list(imp.keys())[:-1], list(imp.values())[:-1], color=color)
    axes[1].set(xlabel="Importance relative", title="Importance des hyperparametres")
    axes[1].grid(axis="x", alpha=.3)
except Exception:
    axes[1].text(.5, .5, "Pas assez de trials", ha="center", transform=axes[1].transAxes)
plt.tight_layout(); savefig(fig, fname); plt.show()

def shap_tree(model, X_sample, names, title, fname_bar, fname_bee):
    import shap
    expl = shap.TreeExplainer(model)
    sv = expl.shap_values(X_sample)
    if isinstance(sv, list):
        # ancien format : liste par classe
        sv = sv[1]
    if getattr(sv, "ndim", 2) == 3:
        # format recent : (n, n_feat, n_classes)
        sv = sv[... , 1]
    fig = plt.figure()
    shap.summary_plot(sv, X_sample, feature_names=names, plot_type="bar", max_display=25, show=False)
    plt.title(f"SHAP (importance globale) – {title}"); plt.tight_layout()
    savefig(plt.gcf(), fname_bar); plt.show()
    fig = plt.figure()
    shap.summary_plot(sv, X_sample, feature_names=names, max_display=25, show=False)
    plt.title(f"SHAP (beeswarm) – {title}"); plt.tight_layout()
    savefig(plt.gcf(), fname_bee); plt.show()
    return sv

# echantillon test commun pour SHAP (reproductible)
_rng = np.random.RandomState(RANDOM_STATE)
_shap_idx = _rng.choice(len(X_test_proc), min(SHAP_SAMPLE, len(X_test_proc)), replace=False)
skf5 = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
print("Fonctions utilitaires pretes.")

```

Fonctions utilitaires pretes.

✧ 8. Modele 2 — Random Forest

Baseline tabulaire interpretable (bagging). Pipeline complet : **Optuna** (AUC en CV 3-fold), **courbe d'apprentissage** (OOB & validation vs nombre d'arbres), **matrice de confusion**, **validation croisee 5-fold**, **analyse SHAP**.

✧ 8.1 — Optimisation Optuna

```

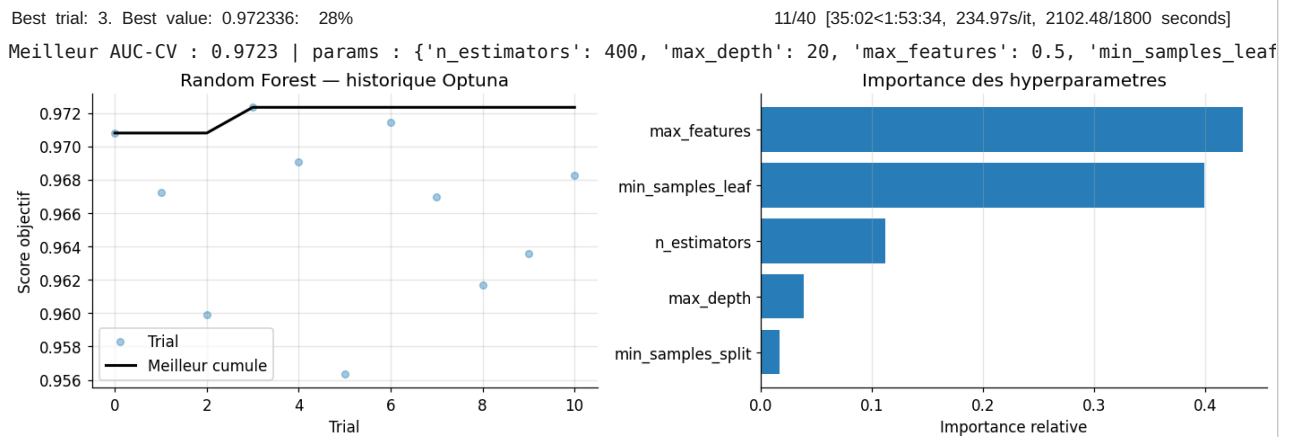
scale_pos = (y_train_bal == 0).sum() / max(1, (y_train_bal == 1).sum())

def objective_rf(trial):
    params = dict(
        n_estimators=trial.suggest_int("n_estimators", 200, 800, step=100),
        max_depth=trial.suggest_categorical("max_depth", [None, 15, 20, 30]),
        max_features=trial.suggest_categorical("max_features", ["sqrt", "log2", 0.3, 0.5]),
        min_samples_leaf=trial.suggest_int("min_samples_leaf", 1, 10),
        min_samples_split=trial.suggest_int("min_samples_split", 2, 15),
        class_weight="balanced", n_jobs=-1, random_state=RANDOM_STATE)
    sc = cross_val_score(RandomForestClassifier(**params), X_train_proc, y_train_bal,
                        cv=StratifiedKFold(3, shuffle=True, random_state=RANDOM_STATE),
                        scoring="roc_auc", n_jobs=1)

    return sc.mean()

study_rf = optuna.create_study(direction="maximize",
                              sampler=optuna.samplers.TPESampler(seed=RANDOM_STATE))
study_rf.enqueue_trial({"n_estimators": 500, "max_depth": None, "max_features": "sqrt",
                       "min_samples_leaf": 2, "min_samples_split": 2})
study_rf.optimize(objective_rf, n_trials=TRIALS["rf"], timeout=OPTUNA_TIMEOUT, show_progress_bar=True)
print("Meilleur AUC-CV :", round(study_rf.best_value, 4), "| params :", study_rf.best_params)
optuna_history_fig(study_rf, "Random Forest", "fig03_optuna_rf.png", color="#2980b9")

```



8.2 — Entrainement final + evaluation

```
best_rf_params = {**study_rf.best_params, "class_weight": "balanced",
                  "oob_score": True, "n_jobs": -1, "random_state": RANDOM_STATE}
rf = RandomForestClassifier(**best_rf_params).fit(X_train_proc, y_train_bal)
print(f"OoB score : {rf.oob_score_:.4f}")
y_pred_rf = rf.predict(X_test_proc)
y_proba_rf = rf.predict_proba(X_test_proc)[: , 1]
m_rf = evaluate("Random Forest", y_test, y_pred_rf, y_proba_rf)
```

OoB score : 0.9325

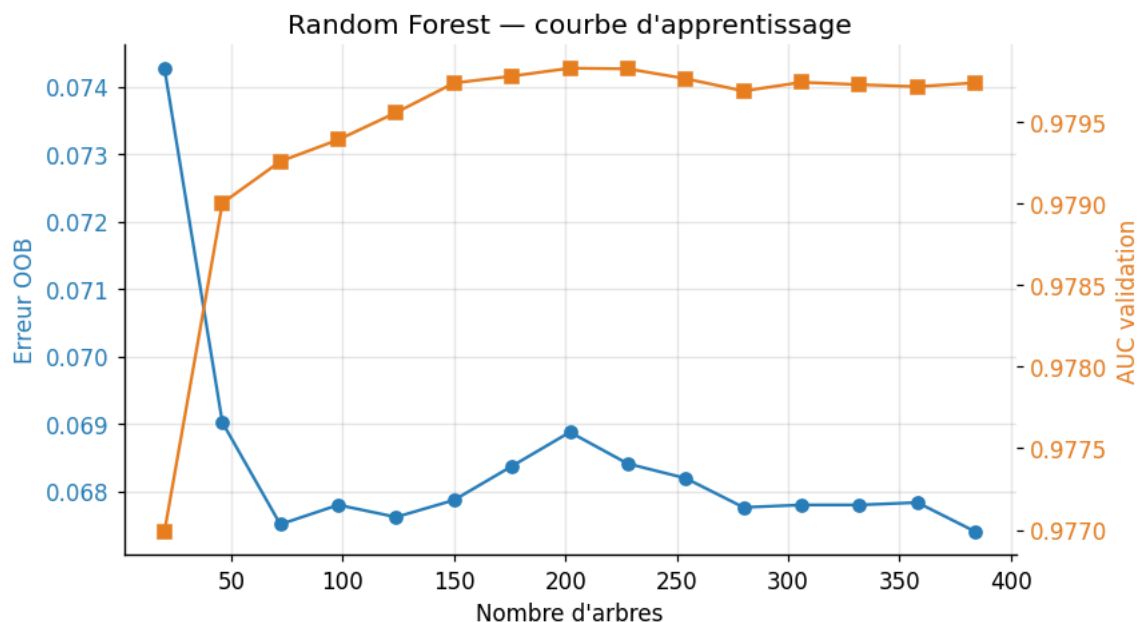
Random Forest

roc_auc	: 0.9788
avg_precision	: 0.9759
recall	: 0.9192
precision	: 0.9282
f1	: 0.9236
balanced_accuracy	: 0.9297
accuracy	: 0.9306

8.3 — Courbe d'apprentissage (OOB & validation vs nombre d'arbres)

Le Random Forest n'a pas de courbe de perte iterative ; on suit a la place l'**erreur OOB** et l'**AUC de validation** quand on ajoute des arbres (`warm_start`) — diagnostic de convergence et de sur-apprentissage.

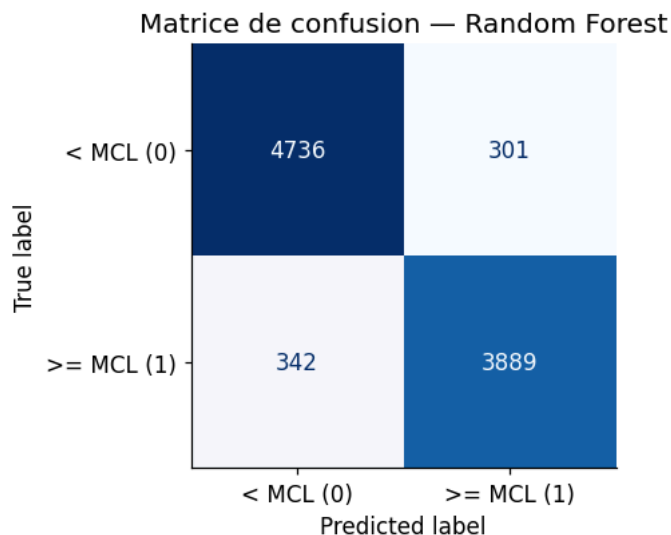
```
stages = list(range(20, int(best_rf_params["n_estimators"]) + 1, max(20, int(best_rf_params["n_estimators"])/15)))
rf_lc = RandomForestClassifier(**best_rf_params, "warm_start": True, "n_estimators": stages[0])
oob_err, val_auc = [], []
for n in stages:
    rf_lc.set_params(n_estimators=n); rf_lc.fit(X_train_proc, y_train_bal)
    oob_err.append(1 - rf_lc.oob_score_)
    val_auc.append(roc_auc_score(y_val, rf_lc.predict_proba(X_val_proc)[: , 1]))
fig, ax1 = plt.subplots(figsize=(8, 4.5))
ax1.plot(stages, oob_err, "o-", color="#2980b9", label="Erreur OOB"); ax1.set_xlabel("Nombre d'arbres")
ax1.set_ylabel("Erreur OOB", color="#2980b9"); ax1.tick_params(axis="y", labelcolor="#2980b9"); ax1.grid(alpha=.5)
ax2 = ax1.twinx(); ax2.plot(stages, val_auc, "s-", color="#e67e22", label="AUC validation")
ax2.set_ylabel("AUC validation", color="#e67e22"); ax2.tick_params(axis="y", labelcolor="#e67e22")
ax1.set_title("Random Forest — courbe d'apprentissage")
plt.tight_layout(); savefig(fig, "fig04_rf_learning_curve.png"); plt.show()
```



8.4 — Validation croisee 5-fold + matrice de confusion

```
cv_rf = cross_validate(RandomForestClassifier(**best_rf_params, "oob_score": False),
                        X_train_proc, y_train_bal, cv=skf5,
                        scoring=["roc_auc", "f1", "recall", "balanced_accuracy"], n_jobs=-1)
for k in ["roc_auc", "f1", "recall", "balanced_accuracy"]:
    print(f" CV {k:18s}: {cv_rf['test_'+k].mean():.4f} ± {cv_rf['test_'+k].std():.4f}")
plot_confusion("Random Forest", y_test, y_pred_rf, "fig05_rf_confusion.png")
```

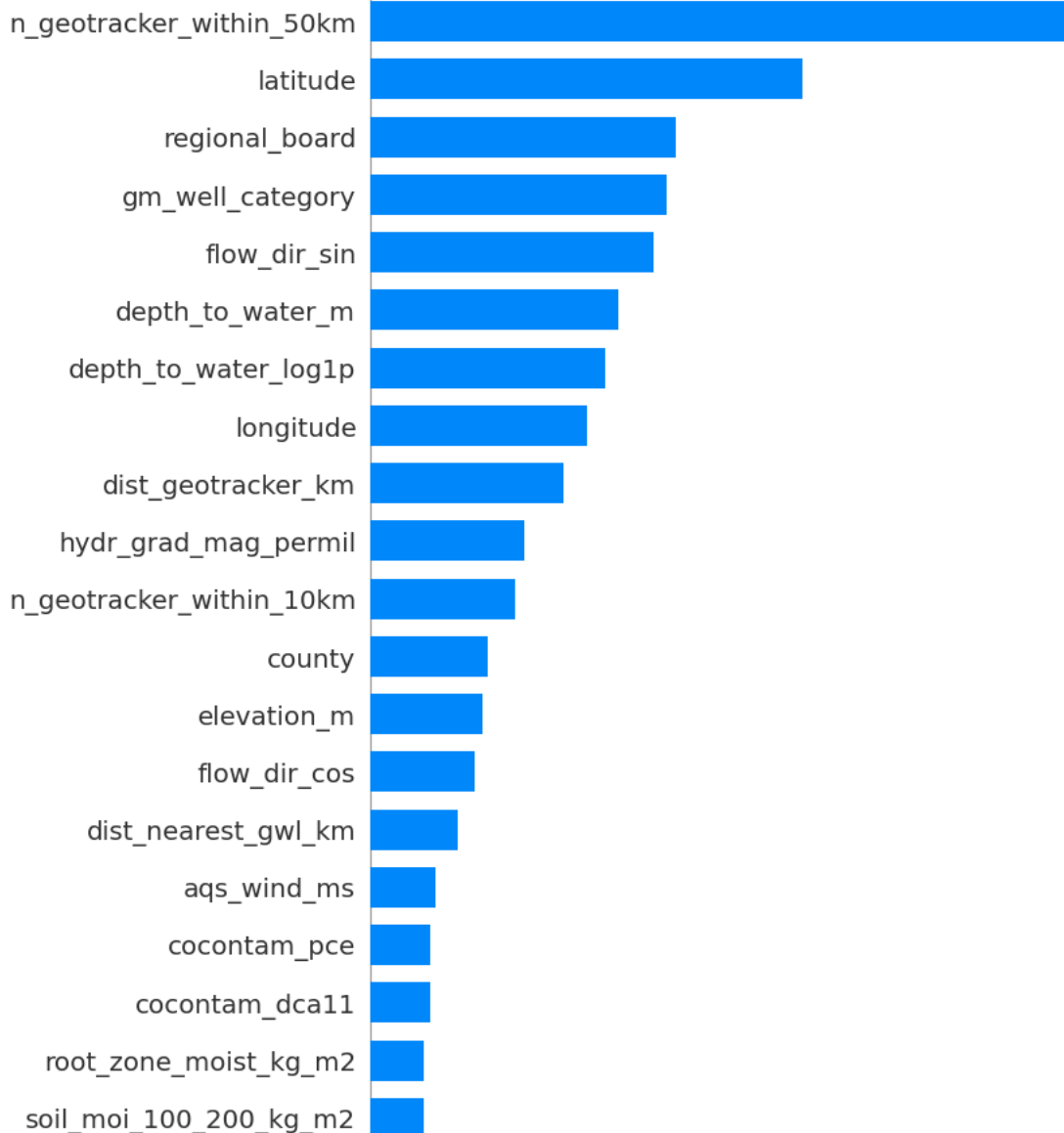
```
CV roc_auc      : 0.9752 ± 0.0011
CV f1           : 0.9206 ± 0.0026
CV recall       : 0.9134 ± 0.0068
CV balanced_accuracy : 0.9269 ± 0.0024
```



8.5 — Analyse SHAP (Random Forest)

```
if RUN_SHAP:
    sv_rf = shap_tree(rf, X_test_proc[_shap_idx], proc_cols, "Random Forest",
                      "fig06_shap_bar_rf.png", "fig07_shap_beeswarm_rf.png")
else:
    print("SHAP desactive (RUN_SHAP=False)")
```


SHAP (importance globale) — Random Forest



9. Modele 3 — XGBoost

Gradient boosting, état de l'art tabulaire. **Optuna** (AUC sur validation interne + early stopping), **courbe d'apprentissage** native (AUC vs iterations), **matrice de confusion**, **analyse SHAP**.

9.1 — Optimisation Optuna

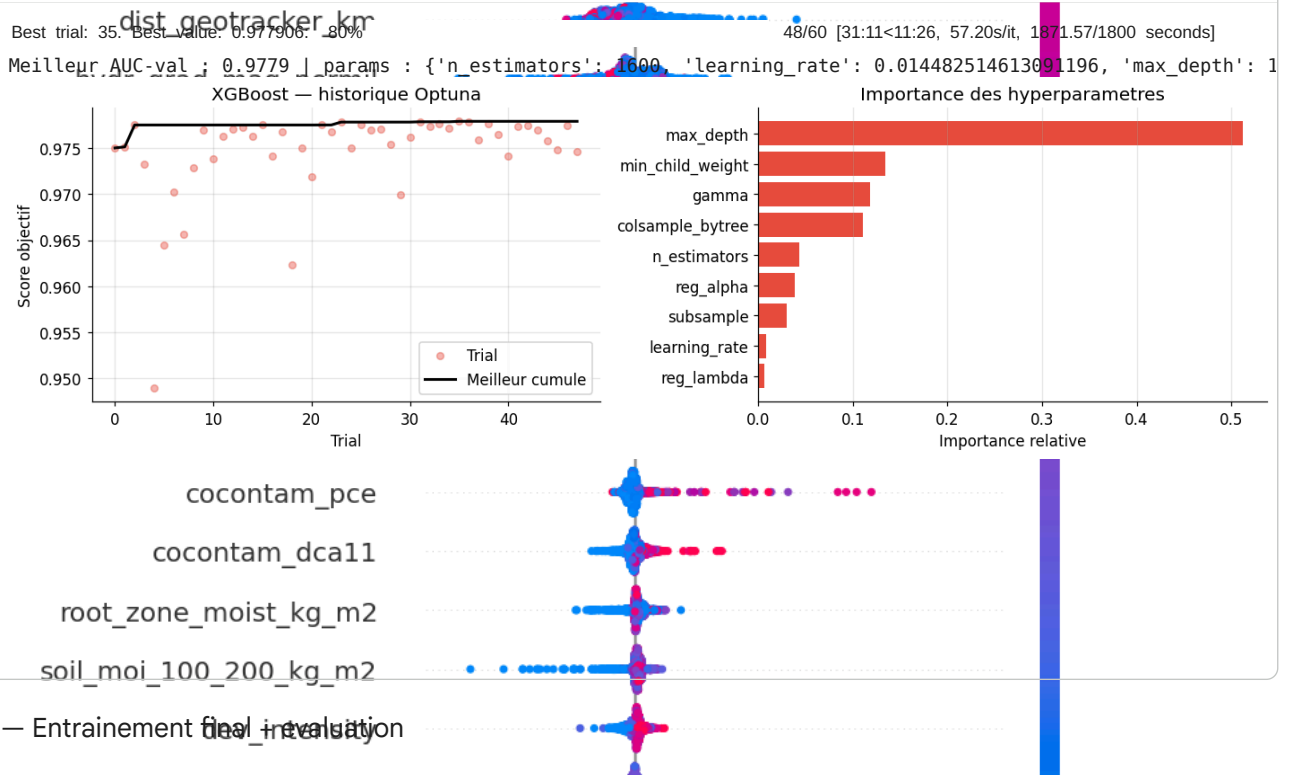
```
def objective_xgb(trial):
    params = dict(
        n_estimators=trial.suggest_int("n_estimators", 200, 2000, step=100),
        learning_rate=trial.suggest_float("learning_rate", 0.005, 0.3, log=True),
        max_depth=trial.suggest_int("max_depth", 3, 12),
        subsample=trial.suggest_float("subsample", 0.5, 1.0),
        colsample_bytree=trial.suggest_float("colsample_bytree", 0.4, 1.0),
        min_child_weight=trial.suggest_int("min_child_weight", 1, 15),
        gamma=trial.suggest_float("gamma", 0.0, 2.0),
        reg_alpha=trial.suggest_float("reg_alpha", 1e-8, 10.0, log=True),
        reg_lambda=trial.suggest_float("reg_lambda", 1e-8, 10.0, log=True),
        scale_pos_weight=scale_pos, tree_method="hist", n_jobs=-1,
        random_state=RANDOM_STATE, eval_metric="auc", verbosity=0)
    model = xgb.XGBClassifier(**params, early_stopping_rounds=40)
    model.fit(X_train_proc, y_train_bal, eval_set=[(X_val_proc, y_val)], verbose=False)
    return roc_auc_score(y_val, model.predict_proba(X_val_proc)[: , 1])

study_xgb = optuna.create_study(direction="maximize",
    sampler=optuna.samplers.TPESampler(seed=RANDOM_STATE),
    pruner=optuna.pruners.MedianPruner(n_startup_trials=5, n_warmup_steps=5))
study_xgb.enqueue_trial({"n_estimators": 1000, "learning_rate": 0.05, "max_depth": 6,
```

```

"subsample": 0.8, "colsample_bytree": 0.8, "min_child_weight": 3, "gamma": 0.1,
"reg_alpha": 0.1, "reg_lambda": 1.0})
study_xgb.optimize(objective_xgb, n_trials=TRIALS["xgb"], timeout=OPTUNA_TIMEOUT, show_progress_bar=True)
print("Meilleur AUC-val :", round(study_xgb.best_value, 4), "| params :", study_xgb.best_params)
optuna_history_fig(study_xgb, "XGBoost", "fig08_optuna_xgb.png", color="#e74c3c")

```



```

best_xgb_params = {**study_xgb.best_params, "scale_pos_weight": scale_pos,
                    "tree_method": "hist", "n_jobs": -1, "random_state": RANDOM_STATE,
                    "eval_metric": "auc", "verbosity": 0}
xgb_model = xgb.XGBClassifier(**best_xgb_params, early_stopping_rounds=50)
xgb_model.fit(X_train_proc, y_train_bal, eval_set=[(X_val_proc, y_val)], verbose=False)
print(f"Best iteration : {xgb_model.best_iteration} | best AUC val : {xgb_model.best_score:.4f}")
y_pred_xgb = xgb_model.predict(X_test_proc)
y_proba_xgb = xgb_model.predict_proba(X_test_proc)[: , 1]
m_xgb = evaluate("XGBoost", y_test, y_pred_xgb, y_proba_xgb)

```

Best iteration : 481 | best AUC val : 0.9779

SHAP value (impact on model output)

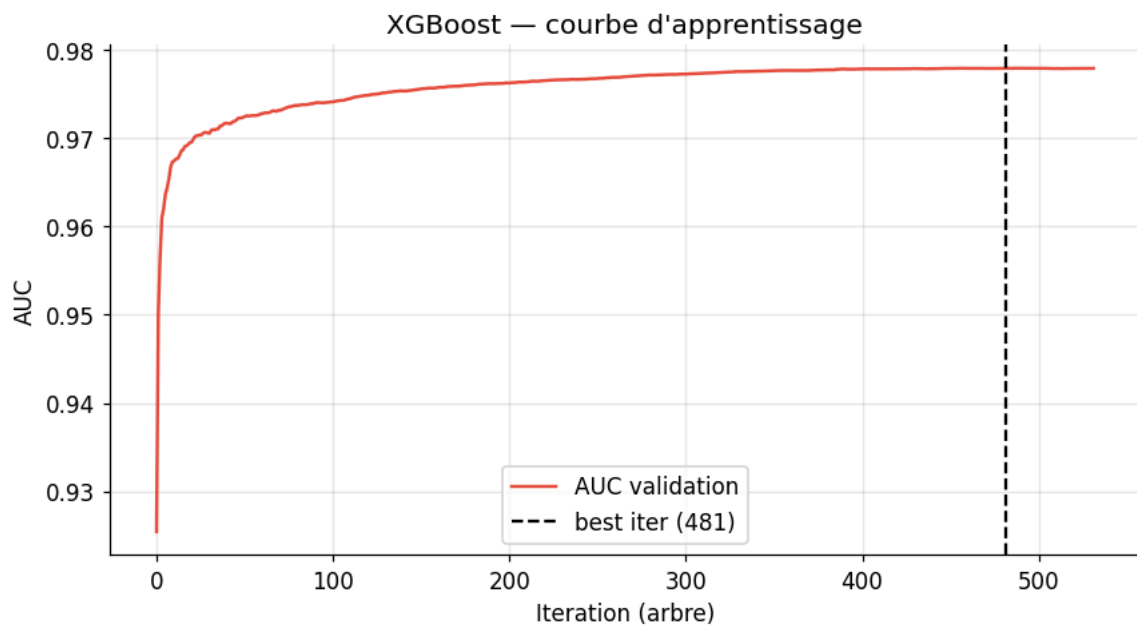
XGBoost	
roc_auc	: 0.9769
avg_precision	: 0.9745
recall	: 0.9211
precision	: 0.9221
f1	: 0.9216
balanced_accuracy	: 0.9279
accuracy	: 0.9285

9.3 — Courbe d'apprentissage native (AUC vs iterations)

```

hist = xgb_model.evals_result()
if hist:
    auc_curve = hist["validation_0"]["auc"]
    fig, ax = plt.subplots(figsize=(8, 4.5))
    ax.plot(auc_curve, color="#e74c3c", lw=1.6, label="AUC validation")
    ax.axvline(xgb_model.best_iteration, color="k", ls="--", label=f"best iter ({xgb_model.best_iteration})")
    ax.set(xlabel="Iteration (arbre)", ylabel="AUC", title="XGBoost — courbe d'apprentissage")
    ax.legend(); ax.grid(alpha=.3)
    plt.tight_layout(); savefig(fig, "fig09_xgb_learning_curve.png"); plt.show()

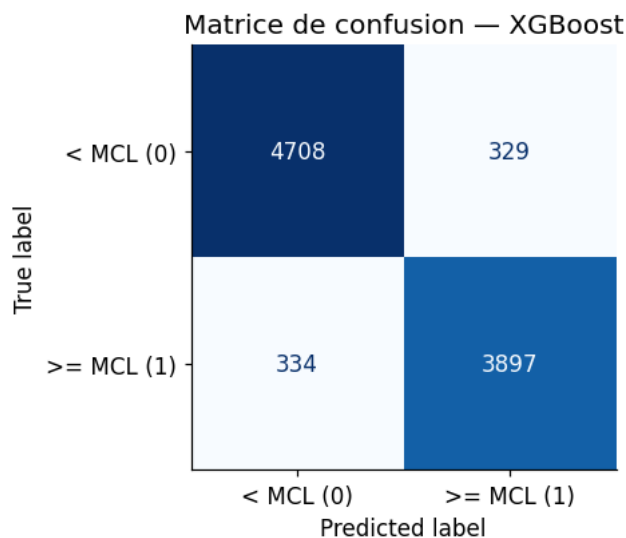
```

9.4 — Validation croisee 5-fold + matrice de confusion

```
cv_xgb = cross_validate(
    xgb.XGBClassifier(**{k: v for k, v in best_xgb_params.items() if k != "eval_metric"}),
    X_train_proc, y_train_bal, cv=skf5,
    scoring=["roc_auc", "f1", "recall", "balanced_accuracy"], n_jobs=-1)
for k in ["roc_auc", "f1", "recall", "balanced_accuracy"]:
    print(f" CV {k:18s}: {cv_xgb['test_'+k].mean():.4f} ± {cv_xgb['test_'+k].std():.4f}")
plot_confusion("XGBoost", y_test, y_pred_xgb, "fig10_xgb_confusion.png")
```

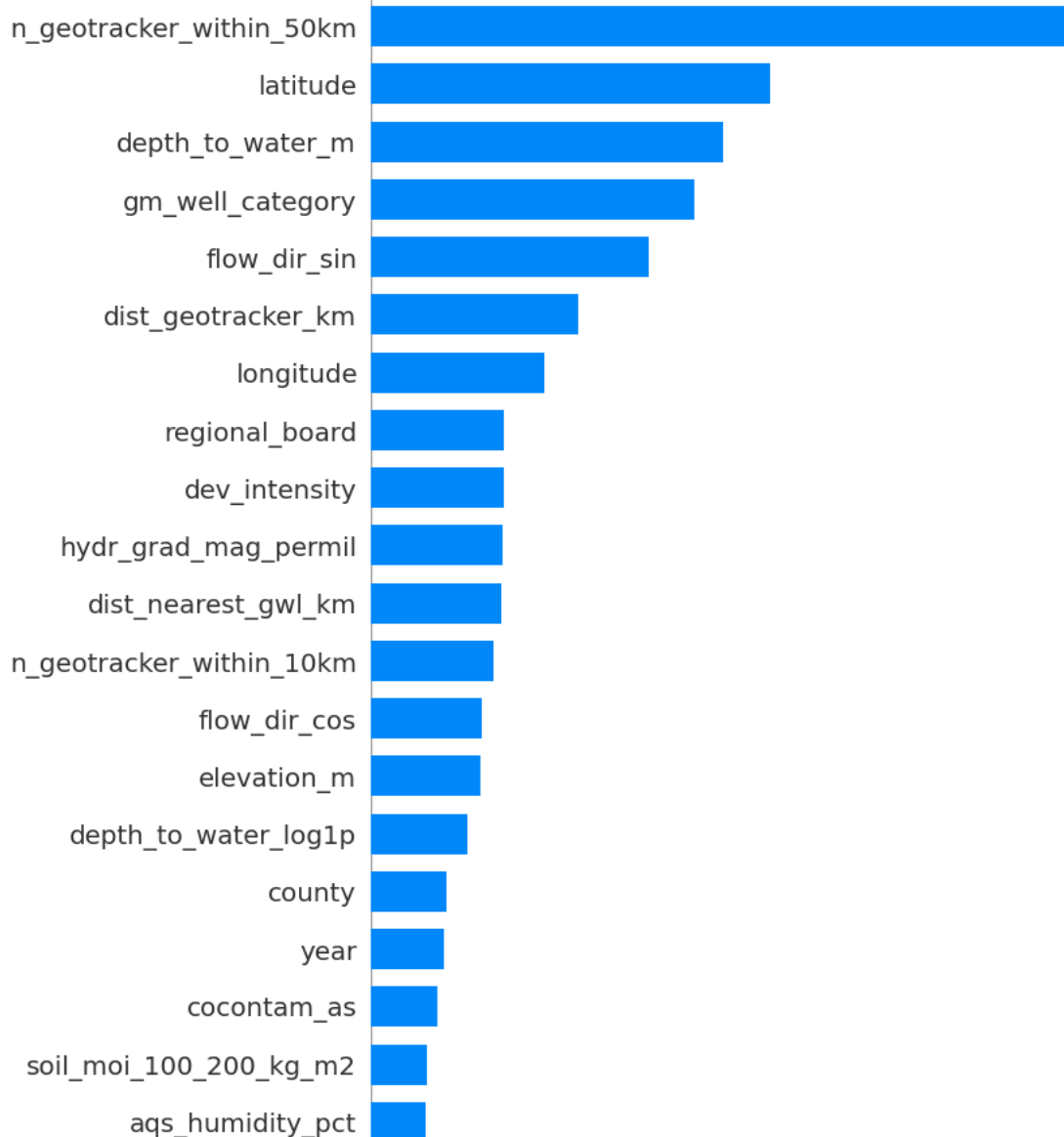
```
CV roc_auc      : 0.9737 ± 0.0014
CV f1           : 0.9192 ± 0.0025
CV recall       : 0.9161 ± 0.0067
CV balanced_accuracy : 0.9256 ± 0.0023
```



9.5 — Analyse SHAP (XGBoost)

```
if RUN_SHAP:
    sv_xgb = shap_tree(xgb_model, X_test_proc[_shap_idx], proc_cols, "XGBoost",
                       "fig11_shap_bar_xgb.png", "fig12_shap_beeswarm_xgb.png")
else:
    print("SHAP desactive (RUN_SHAP=False)")
```


SHAP (importance globale) — XGBoost



10. Modele 1 — HGT (Heterogeneous Graph Transformer)

Coeur du travail de recherche. On construit un **graphe heterogene** ou chaque puits (noeud (`sample`)) est relie a des **noeuds-attributs typés par groupe thematique** (sol, hydrogeologie, proximite des sites, qualite de l'eau, air, meteo...) et a un **cluster géographique** partage (`geo_cluster`), issu d'un KMeans sur lat/lon **ajuste sur le train uniquement** — pas de fuite). Le `HGTConv` apprend des poids et attention spécifiques a chaque type de relation, ce qui distingue le HGT d'un GNN homogene.

Anti-fuite graphe: le KMeans est `fit` sur le train puis `predict` sur val/test. Les noeuds-attributs sont des sous-ensembles standardises des features (aucune concentration PFAS).

10.1 — Construction du graphe hetero (SHAP value) (average impact on model output magnitu

```
# DataFrames standardisees (colonnes nommees) pour piloter la construction du graphe
Xtr_sc_df = pd.DataFrame(X_train_sc, columns=proc_cols)
Xva_sc_df = pd.DataFrame(X_val_sc, columns=proc_cols)
Xte_sc_df = pd.DataFrame(X_test_sc, columns=proc_cols)

GRAPH_GROUPS = {g: feats for g, feats in FEATURE_GROUPS.items()} # 1 type de noeud / groupe
LATLON = [c for c in ["latitude", "longitude"] if c in proc_cols]

def fit_geo_kmeans(Xsc_df, n_clusters):
    if not LATLON:
        return None
    coords = Xsc_df[LATLON].values.astype(np.float32)
    k = min(n_clusters, max(2, len(Xsc_df)//50))
    return KMeans(n_clusters=k, random_state=RANDOM_STATE, n_init=10).fit(coords)
```

```
def build_graph(Xsc_df, y_arr, groups, kmeans):
    data = HeteroData(); n = len(Xsc_df)
    data["sample"].x = torch.FloatTensor(Xsc_df.values)
    data["sample"].y = torch.LongTensor(np.asarray(y_arr))
    eye = torch.stack([torch.arange(n), torch.arange(n)], dim=0)
    for g, feats in groups.items():
        feats = [f for f in feats if f in Xsc_df.columns]
        if not feats or g == "Geospatial": # lat/lon servant au geo_cluster
            continue
        data[g].x = torch.FloatTensor(Xsc_df[feats].values)
        data["sample", f"has_{g}", g].edge_index = eye
        data[g, f"in_{g}", "sample"].edge_index = eye
    if kmeans is not None:
        lab = kmeans.predict(Xsc_df[LATLON].values.astype(np.float32))
        data["geo_cluster"].x = torch.FloatTensor(kmeans.cluster_centers_.astype(np.float32))
        src = np.arange(n)
        data["sample", "located_in", "geo_cluster"].edge_index = torch.tensor([src, lab], dtype=torch.long)
        data["geo_cluster", "contains", "sample"].edge_index = torch.tensor([lab, src], dtype=torch.long)
    return data

if TORCH_GEOMETRIC_AVAILABLE:
    geo_km = fit_geo_kmeans(Xtr_sc_df, N_GEO_CLUSTERS)
    train_data = build_graph(Xtr_sc_df, y_train_bal, GRAPH_GROUPS, geo_km)
    val_data = build_graph(Xva_sc_df, y_val, GRAPH_GROUPS, geo_km)
    test_data = build_graph(Xte_sc_df, y_test, GRAPH_GROUPS, geo_km)
    print("Types de noeuds :", list(train_data.node_types))
    print("Types d'aretes :", len(train_data.edge_types))
else:
    print("PyTorch Geometric requis pour le HGT.")
```

```
Types de noeuds : ['sample', 'Facility_Proximity', 'Water_Quality_cocontam', 'Soil_SSURGO', 'Air_AQS', 'Environme']
Types d'aretes : 16
```

10.2 — Architecture HGT

```
class HGT_Classifier(nn.Module):
    # HGT pour classification binaire ; renvoie logits ou embeddings du noeud 'sample'.
    def __init__(self, metadata, hidden_channels=64, out_channels=2,
                  num_heads=4, num_layers=2, dropout=0.3):
        super().__init__()
        self.lin_dict = nn.ModuleDict({nt: PyGLinear(-1, hidden_channels) for nt in metadata[0]})
        self.convs = nn.ModuleList([
            HGTConv(hidden_channels, hidden_channels, metadata, heads=num_heads)
            for _ in range(num_layers)])
        self.classifier = nn.Sequential(
            nn.Linear(hidden_channels, hidden_channels // 2), nn.ReLU(), nn.Dropout(dropout),
            nn.Linear(max(1, hidden_channels // 2), out_channels))

    def forward(self, x_dict, edge_index_dict, return_embeddings=False):
        x_dict = {nt: self.lin_dict[nt](x).relu() for nt, x in x_dict.items()}
        for conv in self.convs:
            x_dict = conv(x_dict, edge_index_dict)
        emb = x_dict["sample"]
        return emb if return_embeddings else self.classifier(emb)

    def graph_tensors(data):
        xd = {nt: data[nt].x.to(device) for nt in data.node_types}
        ed = {et: data[et].edge_index.to(device) for et in data.edge_types}
        return xd, ed

if TORCH_GEOMETRIC_AVAILABLE:
    metadata = train_data.metadata()
    _m0 = HGT_Classifier(metadata).to(device)
    with torch.no_grad():
        _xd, _ed = graph_tensors(train_data); _ = _m0(_xd, _ed)
    print(f"HGT initialise - {sum(p.numel() for p in _m0.parameters()):,} parametres")
```

HGT initialise - 380,660 parametres

10.3 — Boucle d'entrainement (augmentation : bruit gaussien + DropEdge)

Selection du meilleur etat sur le **F1 de validation** (jamais sur le test). Pondération des classes pour gerer le desequilibre residuel.

```
def augment_graph(xd, ed, noise=0.01, drop=0.1):
    xa = {nt: x + noise * torch.randn_like(x) for nt, x in xd.items()}
    ea = {}
    for et, ei in ed.items():
        keep = (torch.rand(ei.size(1), device=ei.device) > drop).nonzero(as_tuple=True)[0]
```

```

    ea[et] = ei if keep.numel() == 0 else ei[:, keep]
    return xa, ea

def train_hgt(model, train_data, val_data, epochs=120, lr=3e-3, weight_decay=1e-5,
              augment=True, noise=0.01, drop=0.1, verbose=True):
    opt = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)
    cc = np.bincount(train_data["sample"].y.numpy(), minlength=2) # toujours 2 classes -> poids coherents (evit
    cw = torch.FloatTensor([1.0/max(1, c) for c in cc]).to(device); cw = cw / cw.sum()
    crit = nn.CrossEntropyLoss(weight=cw)
    sch = ReduceLROnPlateau(opt, mode="max", factor=0.5, patience=10)
    txd, ted = graph_tensors(train_data); ty = train_data["sample"].y.to(device)
    vxd, ved = graph_tensors(val_data); vy = val_data["sample"].y.to(device)
    best, best_state = 0.0, None
    h = {"train_loss": [], "train_acc": [], "val_acc": [], "val_f1": []}
    for ep in range(1, epochs+1):
        model.train(); opt.zero_grad()
        xa, ea = augment_graph(txd, ted, noise, drop) if augment else (txd, ted)
        out = model(xa, ea); loss = crit(out, ty); loss.backward(); opt.step()
        model.eval()
        with torch.no_grad():
            vout = model(vxd, ved); vpred = vout.argmax(1)
            vf1 = f1_score(vy.cpu(), vpred.cpu())
            vacc = (vpred == vy).float().mean().item()
        sch.step(vf1)
        h["train_loss"].append(loss.item()); h["train_acc"].append((out.argmax(1)==ty).float().mean().item())
        h["val_acc"].append(vacc); h["val_f1"].append(vf1)
        if vf1 > best:
            best = vf1; best_state = {k: v.clone() for k, v in model.state_dict().items()}
        if verbose and (ep % 10 == 0 or ep == 1):
            print(f" epoch {ep:3d}/{epochs} | loss {loss:.4f} | val_acc {vacc:.4f} | val_f1 {vf1:.4f}")
    if best_state:
        model.load_state_dict(best_state)
    if verbose:
        print(f"Termine – meilleur val F1 : {best:.4f}")
    return model, h

```

✓ 10.4 — Optimisation Optuna du HGT (lent — desactivable via le budget FAST)

```

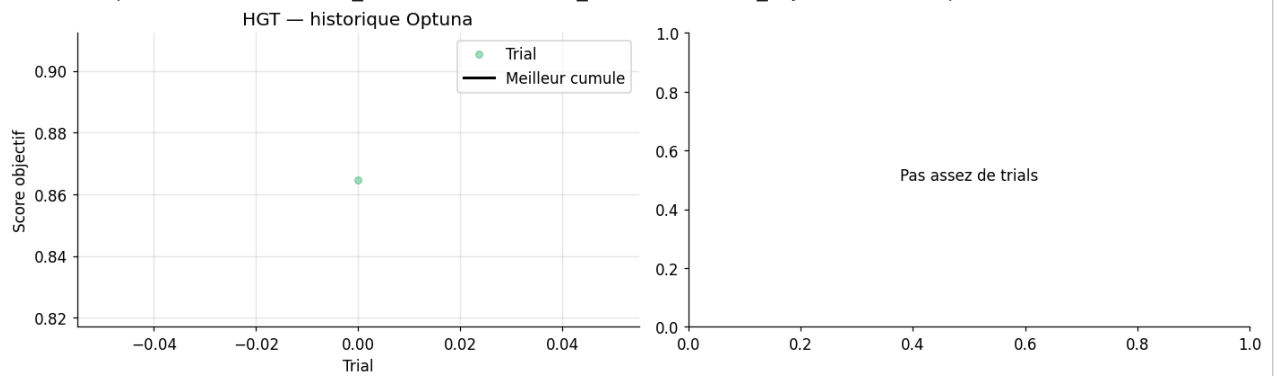
if TORCH_GEOMETRIC_AVAILABLE:
    def objective_hgt(trial):
        m = HGT_Classifier(metadata,
                            hidden_channels=trial.suggest_categorical("hidden_channels", [32, 64, 128]),
                            num_heads=trial.suggest_categorical("num_heads", [2, 4]),
                            num_layers=trial.suggest_int("num_layers", 1, 3),
                            dropout=trial.suggest_float("dropout", 0.1, 0.5)).to(device)
        _, h = train_hgt(m, train_data, val_data, epochs=max(40, EPOCHS_HGT//2),
                        lr=trial.suggest_float("lr", 1e-4, 1e-2, log=True),
                        weight_decay=trial.suggest_float("weight_decay", 1e-5, 1e-3, log=True),
                        verbose=False)
        return max(h["val_f1"])
    study_hgt = optuna.create_study(direction="maximize",
                                    sampler=optuna.samplers.TPESampler(seed=RANDOM_STATE))
    study_hgt.enqueue_trial({"hidden_channels": 64, "num_heads": 4, "num_layers": 2,
                            "dropout": 0.3, "lr": 3e-3, "weight_decay": 1e-5})
    study_hgt.optimize(objective_hgt, n_trials=TRIALS["hgt"], timeout=OPTUNA_TIMEOUT, show_progress_bar=True)
    bp = study_hgt.best_params
    print("Meilleurs params HGT :", bp)
    optuna_history_fig(study_hgt, "HGT", "fig13_optuna_hgt.png", color="#27ae60")

```

Best trial: 0. Best value: 0.864718: 7%

1/15 [1:05:30<15:17:00, 3930.05s/it, 3930.03/1800 seconds]

Meilleurs params HGT : {'hidden_channels': 64, 'num_heads': 4, 'num_layers': 2, 'dropout': 0.3, 'lr': 0.003, 'wei



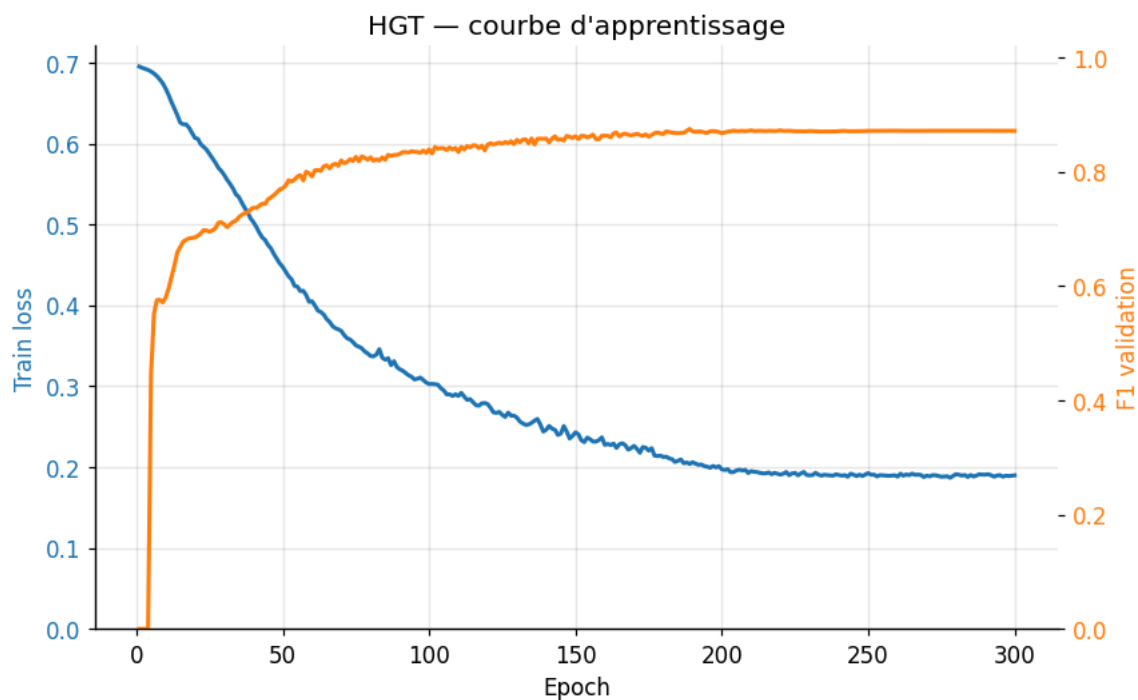
10.5 — Entrainement final + courbe d'apprentissage

```
if TORCH_GEOMETRIC_AVAILABLE:
    hgt_model = HGT_Classifier(metadata, hidden_channels=bp["hidden_channels"],
                                num_heads=bp["num_heads"], num_layers=bp["num_layers"], dropout=bp["dropout"]).to(device)
    with torch.no_grad():
        _xd, _ed = graph_tensors(train_data); _ = hgt_model(_xd, _ed)
    hgt_model, hgt_hist = train_hgt(hgt_model, train_data, val_data, epochs=EPOCHS_HGT,
                                    lr=bp["lr"], weight_decay=bp["weight_decay"], verbose=True)

    ep = range(1, len(hgt_hist["train_loss"])+1)
    fig, ax1 = plt.subplots(figsize=(8, 5))
    ax1.plot(ep, hgt_hist["train_loss"], color="tab:blue", lw=2, label="Train loss")
    ax1.set_xlabel("Epoch"); ax1.set_ylabel("Train loss", color="tab:blue")
    ax1.tick_params(axis="y", labelcolor="tab:blue"); ax1.grid(alpha=.3); ax1.set_ylim(bottom=0)
    ax2 = ax1.twinx(); ax2.plot(ep, hgt_hist["val_f1"], color="tab:orange", lw=2, label="Val F1")
    ax2.set_ylabel("F1 validation", color="tab:orange"); ax2.tick_params(axis="y", labelcolor="tab:orange")
    ax2.set_ylim(0, 1.02); ax1.set_title("HGT — courbe d'apprentissage")
    plt.tight_layout(); savefig(fig, "fig14_hgt_learning_curve.png"); plt.show()
```

epoch	1/300	loss	0.6951	val_acc	0.5435	val_f1	0.0000
epoch	10/300	loss	0.6679	val_acc	0.6472	val_f1	0.5797
epoch	20/300	loss	0.6069	val_acc	0.6836	val_f1	0.6849
epoch	30/300	loss	0.5611	val_acc	0.7284	val_f1	0.7079
epoch	40/300	loss	0.5026	val_acc	0.7646	val_f1	0.7369
epoch	50/300	loss	0.4474	val_acc	0.8005	val_f1	0.7710
epoch	60/300	loss	0.4050	val_acc	0.8227	val_f1	0.7918
epoch	70/300	loss	0.3682	val_acc	0.8375	val_f1	0.8196
epoch	80/300	loss	0.3373	val_acc	0.8425	val_f1	0.8252
epoch	90/300	loss	0.3208	val_acc	0.8473	val_f1	0.8310
epoch	100/300	loss	0.3026	val_acc	0.8529	val_f1	0.8391
epoch	110/300	loss	0.2879	val_acc	0.8562	val_f1	0.8344
epoch	120/300	loss	0.2778	val_acc	0.8589	val_f1	0.8474
epoch	130/300	loss	0.2610	val_acc	0.8661	val_f1	0.8557
epoch	140/300	loss	0.2462	val_acc	0.8663	val_f1	0.8580
epoch	150/300	loss	0.2430	val_acc	0.8728	val_f1	0.8621
epoch	160/300	loss	0.2274	val_acc	0.8746	val_f1	0.8631
epoch	170/300	loss	0.2262	val_acc	0.8735	val_f1	0.8649
epoch	180/300	loss	0.2125	val_acc	0.8786	val_f1	0.8661
epoch	190/300	loss	0.2059	val_acc	0.8816	val_f1	0.8706
epoch	200/300	loss	0.1972	val_acc	0.8784	val_f1	0.8675
epoch	210/300	loss	0.1949	val_acc	0.8824	val_f1	0.8724
epoch	220/300	loss	0.1907	val_acc	0.8828	val_f1	0.8724
epoch	230/300	loss	0.1898	val_acc	0.8812	val_f1	0.8711
epoch	240/300	loss	0.1901	val_acc	0.8814	val_f1	0.8709
epoch	250/300	loss	0.1925	val_acc	0.8814	val_f1	0.8712
epoch	260/300	loss	0.1882	val_acc	0.8820	val_f1	0.8716
epoch	270/300	loss	0.1894	val_acc	0.8819	val_f1	0.8715
epoch	280/300	loss	0.1912	val_acc	0.8819	val_f1	0.8715
epoch	290/300	loss	0.1907	val_acc	0.8817	val_f1	0.8714
epoch	300/300	loss	0.1897	val_acc	0.8819	val_f1	0.8715

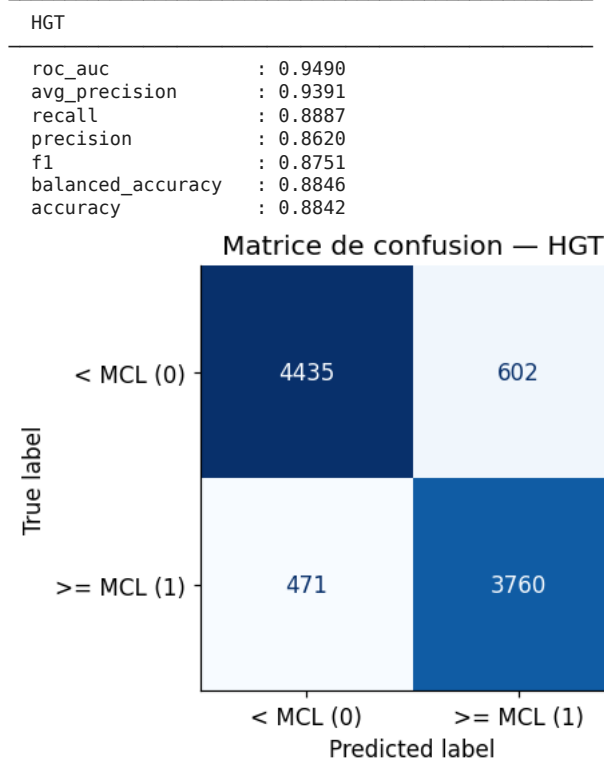
Termine – meilleur val F1 : 0.8748



10.6 — Evaluation + matrice de confusion

```
def hgt_predict(model, data):
    model.eval()
    with torch.no_grad():
        xd, ed = graph_tensors(data)
        return F.softmax(model(xd, ed), dim=1).cpu().numpy()

if TORCH_GEOMETRIC_AVAILABLE:
    hgt_out = hgt_predict(hgt_model, test_data)
    y_pred_hgt = hgt_out.argmax(1); y_proba_hgt = hgt_out[:, 1]
    m_hgt = evaluate("HGT", y_test, y_pred_hgt, y_proba_hgt)
    plot_confusion("HGT", y_test, y_pred_hgt, "fig15_hgt_confusion.png")
```



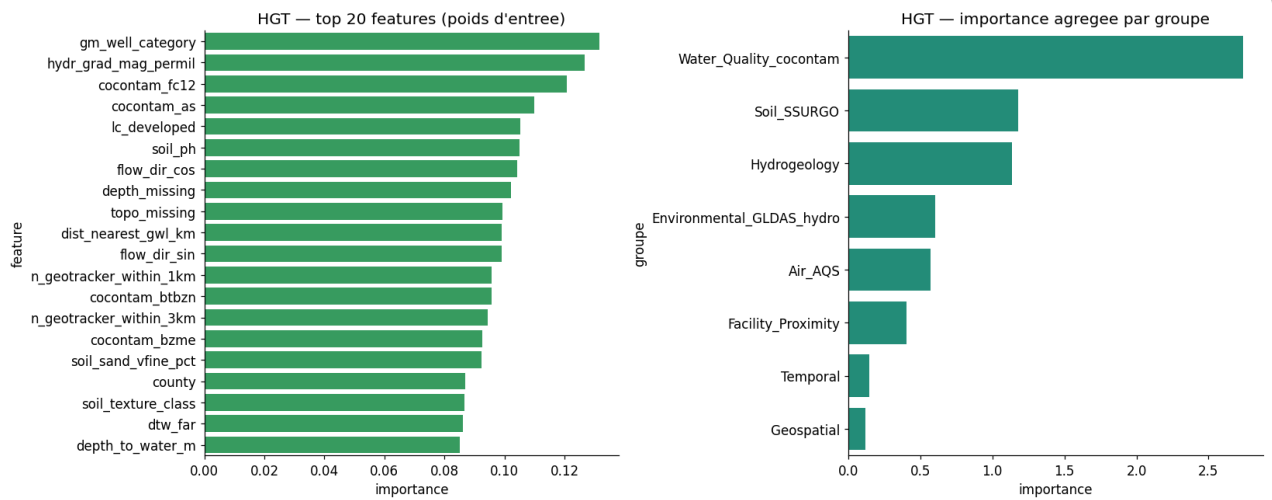
10.7 — Explicabilite du HGT

Sur un graphe transductif, le Tree/Kernel-SHAP classique est mal defini. On utilise donc **deux attributions complementaires** :

1. **shap.GradientExplainer** (expected gradients) — variante SHAP pour modeles differentiables, appliquee aux features du noeud `sample` (graphe fixe) ;
2. **Importance par groupe** — norme L1 des poids de la couche d'entree `lin_dict['sample']`, agregee par groupe thematique (lecture macro du signal mobilise).

```
if TORCH_GEOMETRIC_AVAILABLE:
    # --- 1) importance des features (poids lineaires d'entree) + agregation par groupe ---
    with torch.no_grad():
        w = hgt_model.lin_dict["sample"].weight.detach().cpu().numpy() # (hidden, n_feat)
        imp_feat = np.abs(w).mean(0)
        hgt_imp = pd.DataFrame({"feature": proc_cols, "importance": imp_feat}).sort_values("importance", ascending=False)
        grp_imp = []
        for g, feats in FEATURE_GROUPS.items():
            fin = [f for f in feats if f in proc_cols]
            if fin:
                grp_imp.append({"groupe": g, "importance": hgt_imp.set_index("feature").loc[fin, "importance"].sum()})
        grp_imp = pd.DataFrame(grp_imp).sort_values("importance", ascending=False)

    fig, ax = plt.subplots(1, 2, figsize=(15, 6))
    sns.barplot(data=hgt_imp.head(20), x="importance", y="feature", color="#27ae60", ax=ax[0])
    ax[0].set_title("HGT - top 20 features (poids d'entree)")
    sns.barplot(data=grp_imp, x="importance", y="groupe", color="#16a085", ax=ax[1])
    ax[1].set_title("HGT - importance agregee par groupe")
    plt.tight_layout(); savefig(fig, "fig16_hgt_importance.png"); plt.show()
```

```
# --- 2) SHAP GradientExplainer (attribution differentiable sur le noeud 'sample') ---
if TORCH_GEOMETRIC_AVAILABLE and RUN_SHAP_HGT:
    import shap
    # indices de colonnes de chaque groupe-noeud dans proc_cols (pour reconstruire les noeuds)
    _grp_node_idx = {g: [proc_cols.index(f) for f in feats if f in proc_cols]
                      for g, feats in GRAPH_GROUPS.items() if g != "Geospatial" and g in test_data.node_types}
    _edge_test = {et: test_data[et].edge_index.to(device) for et in test_data.edge_types}
    _centers = test_data["geo_cluster"].x.to(device) if "geo_cluster" in test_data.node_types else None

    class HGTGrad(nn.Module):
        # Fonction differentiable sample_x -> logits, graphe (aretes + clusters) fixe.
        def __init__(self, model):
            super().__init__(); self.m = model
        def forward(self, sx):
            xd = {"sample": sx}
            for g, idx in _grp_node_idx.items():
                xd[g] = sx[:, idx]
            if _centers is not None:
                xd["geo_cluster"] = _centers
            xd = {nt: self.m.lin_dict[nt](x).relu() for nt, x in xd.items()}
            for conv in self.m.convs:
                xd = conv(xd, _edge_test)
            return self.m.classifier(xd["sample"])

    try:
        wrap = HGTGrad(hgt_model).to(device).eval()
        Xte_t = test_data["sample"].x.to(device)
        n_bg, n_ex = min(100, len(Xte_t)), min(300, len(Xte_t))
        bg = Xte_t[np.random.RandomState(RANDOM_STATE).choice(len(Xte_t), n_bg, replace=False)]
        ex_idx = np.random.RandomState(1).choice(len(Xte_t), n_ex, replace=False)
        explainer = shap.GradientExplainer(wrap, bg)
        sv = explainer.shap_values(Xte_t[ex_idx])
        sv1 = sv[1] if isinstance(sv, list) else sv          # classe positive
        sv1 = sv1[... , 1] if sv1.ndim == 3 else sv1
        fig = plt.figure()
        shap.summary_plot(sv1, Xte_t[ex_idx].cpu().numpy(), feature_names=proc_cols,
                          plot_type="bar", max_display=25, show=False)
        plt.title("SHAP (GradientExplainer) - HGT"); plt.tight_layout()
        savefig(plt.gcf(), "fig17_shap_hgt.png"); plt.show()
    except Exception as e:
        print("GradientExplainer indisponible, repli sur l'importance par poids :", e)
```

GradientExplainer indisponible, repli sur l'importance par poids : Found indices in 'edge_index' that are larger

11. Modele 4 — Embedding Fusion (HGT -> XGBoost)

Strategie hybride : 1) extraire les **embeddings** appris par le HGT (representation du noeud `sample` apres message-passing), 2) les reduire par **PCA (95 % de variance)**, 3) les **concatener** aux features tabulaires, 4) entrainer un **XGBoost** (optimise Optuna) sur cette

representation enrichie. L'idée : injecter le signal **relationnel** du graphe dans le meilleur modele tabulaire.

✓ 11.1 — Extraction des embeddings + PCA + fusion

```
if TORCH_GEOMETRIC_AVAILABLE:
    def hgt_embeddings(model, data):
        model.eval()
        with torch.no_grad():
            xd, ed = graph_tensors(data)
            return model(xd, ed, return_embeddings=True).cpu().numpy()

    emb_tr = hgt_embeddings(hgt_model, train_data)
    emb_va = hgt_embeddings(hgt_model, val_data)
    emb_te = hgt_embeddings(hgt_model, test_data)
    pca_fus = PCA(n_components=0.95, random_state=RANDOM_STATE)
    emb_tr_p = pca_fus.fit_transform(emb_tr)
    emb_va_p = pca_fus.transform(emb_va)
    emb_te_p = pca_fus.transform(emb_te)
    n_emb = emb_tr_p.shape[1]
    print(f"Embeddings HGT {emb_tr.shape[1]}D -> PCA {n_emb}D "
          f"(variance {pca_fus.explained_variance_ratio_.sum():.1%})")

    Xtr_fus = np.concatenate([X_train_proc, emb_tr_p], axis=1)
    Xva_fus = np.concatenate([X_val_proc, emb_va_p], axis=1)
    Xte_fus = np.concatenate([X_test_proc, emb_te_p], axis=1)
    fusion_names = proc_cols + [f"HGT_emb_{j}" for j in range(n_emb)]
    print(f"Matrice fusion : {Xtr_fus.shape} ({len(proc_cols)} features + {n_emb} embeddings)")

Embeddings HGT 64D -> PCA 3D (variance 95.0%)
Matrice fusion : (27802, 106) (103 features + 3 embeddings)
```

✓ 11.2 — Optimisation Optuna + entraînement

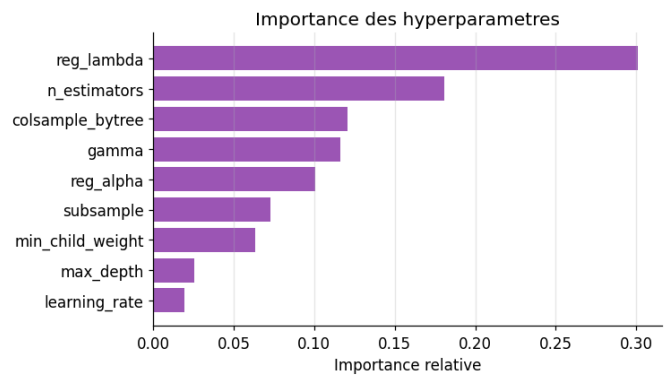
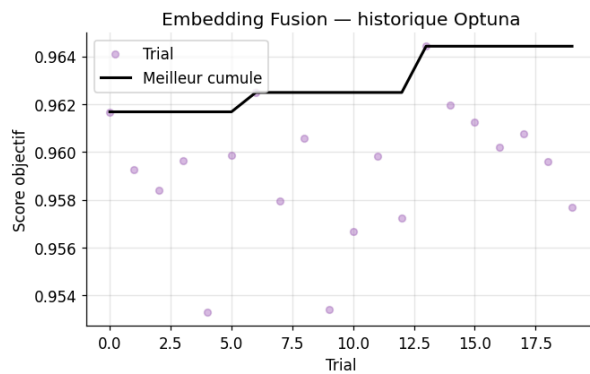
```
if TORCH_GEOMETRIC_AVAILABLE:
    def objective_fusion(trial):
        p = dict(n_estimators=trial.suggest_int("n_estimators", 150, 800),
                  max_depth=trial.suggest_int("max_depth", 3, 12),
                  learning_rate=trial.suggest_float("learning_rate", 0.01, 0.3, log=True),
                  subsample=trial.suggest_float("subsample", 0.5, 1.0),
                  colsample_bytree=trial.suggest_float("colsample_bytree", 0.4, 1.0),
                  min_child_weight=trial.suggest_int("min_child_weight", 1, 10),
                  gamma=trial.suggest_float("gamma", 0.0, 1.0),
                  reg_alpha=trial.suggest_float("reg_alpha", 1e-4, 1.0, log=True),
                  reg_lambda=trial.suggest_float("reg_lambda", 1e-4, 2.0, log=True),
                  scale_pos_weight=scale_pos, tree_method="hist",
                  random_state=RANDOM_STATE, eval_metric="auc", verbosity=0)
        c = xgb.XGBClassifier(**p, early_stopping_rounds=40)
        c.fit(Xtr_fus, y_train_bal, eval_set=[(Xva_fus, y_val)], verbose=False)
        return roc_auc_score(y_val, c.predict_proba(Xva_fus)[: , 1])

    study_fus = optuna.create_study(direction="maximize",
                                     sampler=optuna.samplers.TPESampler(seed=RANDOM_STATE))
    study_fus.optimize(objective_fusion, n_trials=TRIALS["fusion"], timeout=OPTUNA_TIMEOUT, show_progress_bar=True,
                       optuna_history_fig(study_fus, "Embedding Fusion", "fig18_optuna_fusion.png", color="#9b59b6"))

    p_fus = {**study_fus.best_params, "scale_pos_weight": scale_pos, "tree_method": "hist",
              "random_state": RANDOM_STATE, "eval_metric": "auc", "verbosity": 0}
    xgb_fusion = xgb.XGBClassifier(**p_fus, early_stopping_rounds=50)
    xgb_fusion.fit(Xtr_fus, y_train_bal, eval_set=[(Xva_fus, y_val)], verbose=False)
    y_pred_fus = xgb_fusion.predict(Xte_fus)
    y_proba_fus = xgb_fusion.predict_proba(Xte_fus)[: , 1]
    m_fus = evaluate("Embedding Fusion", y_test, y_pred_fus, y_proba_fus)
    plot_confusion("Embedding Fusion", y_test, y_pred_fus, "fig19_fusion_confusion.png")
```

Best trial: 13. Best value: 0.964424: 100%

20/20 [03:35<00:00, 10.69s/it, 215.09/1800 seconds]



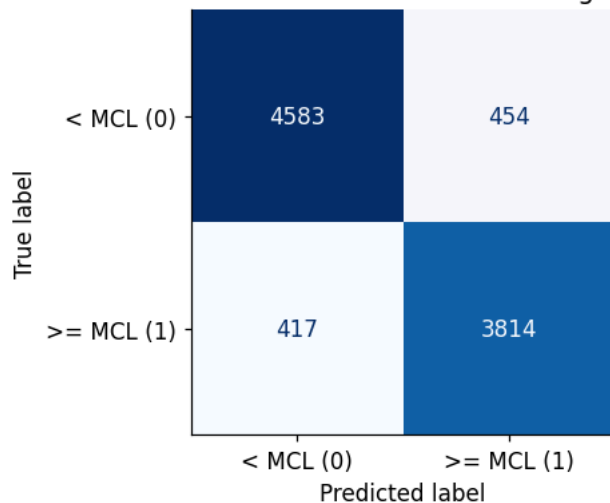
Embedding Fusion

```

roc_auc      : 0.9664
avg_precision : 0.9633
recall       : 0.9014
precision    : 0.8936
f1           : 0.8975
balanced_accuracy : 0.9057
accuracy     : 0.9060

```

Matrice de confusion — Embedding Fusion



11.3 — SHAP : contribution des features tabulaires vs embeddings HGT

```

if TORCH_GEOMETRIC_AVAILABLE and RUN_SHAP:
    import shap
    expl = shap.TreeExplainer(xgb_fusion)
    sv_fus = expl.shap_values(Xte_fus[_shap_idx])
    if isinstance(sv_fus, list):
        sv_fus = sv_fus[1]
    if getattr(sv_fus, "ndim", 2) == 3:
        sv_fus = sv_fus[..., 1]
    fig = plt.figure()
    shap.summary_plot(sv_fus, Xte_fus[_shap_idx], feature_names=fusion_names,
                      plot_type="bar", max_display=25, show=False)
    plt.title("SHAP - Embedding Fusion"); plt.tight_layout()
    savefig(plt.gcf(), "fig20_shap_fusion.png"); plt.show()

    imp = np.abs(sv_fus).mean(0)
    n0 = len(proc_cols)
    orig, embc = imp[:n0].sum(), imp[n0:].sum()
    print(f"Contribution SHAP - features tabulaires : {100*orig/(orig+embc):.1f}% | "
          f"embeddings HGT : {100*embc/(orig+embc):.1f}%")

```

12. Modele 5 — Stacking Ensemble (HGT + XGBoost + RandomForest)

Meta-apprentissage : on combine les predictions des **trois modeles de base** (HGT, XGBoost, RandomForest) en **meta-features enrichies** (probabilites, entropies, confiances, accords inter-modeles, produits/differences) auxquelles on ajoute les **embeddings HGT (PCA 90 %)**. Un **meta-classifieur XGBoost** (optimise Optuna) apprend a arbitrer entre les modeles selon le contexte.

12.1 — Probabilites des modeles de base + meta-features

```
if TORCH_GEOMETRIC_AVAILABLE:
    # probas des 3 modeles de base (train/val/test) - 2 colonnes (classe 0 et 1)
    hgt_p_tr, hgt_p_va, hgt_p_te = (hgt_predict(hgt_model, d) for d in (train_data, val_data, test_data))
    xgb_p_tr = xgb_model.predict_proba(X_train_proc); xgb_p_va = xgb_model.predict_proba(X_val_proc); xgb_p_te =
    rf_p_tr = rf.predict_proba(X_train_proc); rf_p_va = rf.predict_proba(X_val_proc); rf_p_te =

    pca_meta = PCA(n_components=0.90, random_state=RANDOM_STATE)
    emb_tr = pca_meta.fit_transform(emb_tr); emb_va = pca_meta.transform(emb_va); emb_te = pca_meta.transform(emb_t
    n_em = emb_tr.shape[1]

    def meta_features(hp, xp, rp, emb):
        eps = 1e-10
        h1, x1, r1 = hp[:, 1:2], xp[:, 1:2], rp[:, 1:2]
        def ent(p): return (-p * np.log(p + eps)).sum(1, keepdims=True)
        agree = lambda a, b: (a.argmax(1) == b.argmax(1)).astype(np.float32).reshape(-1, 1)
        parts = [h1, x1, r1, ent(hp), ent(xp), ent(rp),
                  hp.max(1, keepdims=True), xp.max(1, keepdims=True), rp.max(1, keepdims=True),
                  agree(hp, xp), agree(hp, rp), agree(xp, rp),
                  np.abs(h1 - x1), np.abs(h1 - r1), np.abs(x1 - r1),
                  h1 * x1, x1 * r1, (h1 + x1 + r1) / 3.0, emb]
        return np.hstack(parts)

    META_BASE = ["hgt_p1", "xgb_p1", "rf_p1", "hgt_ent", "xgb_ent", "rf_ent",
                  "hgt_conf", "xgb_conf", "rf_conf", "agree_hx", "agree_hr", "agree_xr",
                  "diff_hx", "diff_hr", "diff_xr", "prod_hx", "prod_xr", "mean_hxr"]
    meta_tr = meta_features(hgt_p_tr, xgb_p_tr, rf_p_tr, emb_tr)
    meta_va = meta_features(hgt_p_va, xgb_p_va, rf_p_va, emb_va)
    meta_te = meta_features(hgt_p_te, xgb_p_te, rf_p_te, emb_te)
    meta_names = META_BASE + [f"emb_pca_{k}" for k in range(n_em)]
    print(f"Meta-features : {meta_tr.shape[1]} dims ({len(META_BASE)} derivees + {n_em} embeddings PCA)")
```

12.2 — Optimisation Optuna du meta-classifieur + entraînement

```
if TORCH_GEOMETRIC_AVAILABLE:
    def objective_stack(trial):
        p = dict(n_estimators=trial.suggest_int("n_estimators", 50, 400),
                  max_depth=trial.suggest_int("max_depth", 2, 7),
                  learning_rate=trial.suggest_float("learning_rate", 0.01, 0.2, log=True),
                  subsample=trial.suggest_float("subsample", 0.5, 1.0),
                  colsample_bytree=trial.suggest_float("colsample_bytree", 0.5, 1.0),
                  min_child_weight=trial.suggest_int("min_child_weight", 1, 8),
                  gamma=trial.suggest_float("gamma", 0.0, 0.5),
                  reg_alpha=trial.suggest_float("reg_alpha", 1e-4, 0.5, log=True),
                  reg_lambda=trial.suggest_float("reg_lambda", 1e-4, 1.0, log=True),
                  scale_pos_weight=scale_pos, tree_method="hist",
                  random_state=RANDOM_STATE, eval_metric="auc", verbosity=0)
        c = xgb.XGBClassifier(**p, early_stopping_rounds=30)
        c.fit(meta_tr, y_train_bal, eval_set=[(meta_va, y_val)], verbose=False)
        return roc_auc_score(y_val, c.predict_proba(meta_va)[: , 1])

    study_stk = optuna.create_study(direction="maximize",
                                     sampler=optuna.samplers.TPESampler(seed=RANDOM_STATE))
    study_stk.optimize(objective_stack, n_trials=TRIALS["stacking"], timeout=OPTUNA_TIMEOUT, show_progress_bar=
    optuna_history_fig(study_stk, "Stacking", "fig21_optuna_stacking.png", color="#34495e")

    p_stk = {**study_stk.best_params, "scale_pos_weight": scale_pos, "tree_method": "hist",
              "random_state": RANDOM_STATE, "eval_metric": "auc", "verbosity": 0}
    meta_clf = xgb.XGBClassifier(**p_stk, early_stopping_rounds=40)
    meta_clf.fit(meta_tr, y_train_bal, eval_set=[(meta_va, y_val)], verbose=False)
    y_pred_stk = meta_clf.predict(meta_te)
    y_proba_stk = meta_clf.predict_proba(meta_te)[: , 1]
    m_stk = evaluate("Stacking", y_test, y_pred_stk, y_proba_stk)
    plot_confusion("Stacking", y_test, y_pred_stk, "fig22_stacking_confusion.png")
```

12.3 — Importance des meta-features (quelle source domine ?)

```

if TORCH_GEOMETRIC_AVAILABLE:
    imp = np.asarray(meta_clf.feature_importances_)
    mdf = pd.DataFrame({"feature": meta_names[:len(imp)], "importance": imp}).sort_values("importance", ascending=False)
    fig, ax = plt.subplots(figsize=(8, 6))
    sns.barplot(data=mdf.head(15), x="importance", y="feature", color="#34495e", ax=ax)
    ax.set_title("Stacking - top 15 meta-features")
    plt.tight_layout(); savefig(fig, "fig23_stacking_importance.png"); plt.show()
    print(mdf.head(10).to_string(index=False))

```

✓ 13. Comparaison des 5 modeles (test set)

Tableau recapitulatif, courbes ROC / Precision-Rappel, calibration, matrices de confusion et tableau de bord. Tous les modeles sont evalues sur **le meme jeu de test**.

```

ORDER = ["HGT", "Random Forest", "XGBoost", "Embedding Fusion", "Stacking"]
ORDER = [m for m in ORDER if m in RESULTS]
COLORS = {"HGT": "#27ae60", "Random Forest": "#2980b9", "XGBoost": "#e74c3c",
          "Embedding Fusion": "#9b59b6", "Stacking": "#34495e"}

comp = pd.DataFrame([
    "Modele": m,
    "ROC-AUC": RESULTS[m]["metrics"]["roc_auc"],
    "AP": RESULTS[m]["metrics"]["avg_precision"],
    "Rappel": RESULTS[m]["metrics"]["recall"],
    "Precision": RESULTS[m]["metrics"]["precision"],
    "F1": RESULTS[m]["metrics"]["f1"],
    "Bal.Acc": RESULTS[m]["metrics"]["balanced_accuracy"],
    "Accuracy": RESULTS[m]["metrics"]["accuracy"],
] for m in ORDER).set_index("Modele")
comp.to_csv(RES_DIR / "comparaison_modeles.csv")
print(comp.round(4).to_string())
best = comp["F1"].idxmax()
print(f"\nMeilleur modele (F1) : {best} (F1={comp.loc[best, 'F1']:.4f}, "
      f"AUC={comp.loc[best, 'ROC-AUC']:.4f})")
comp.round(4).style.background_gradient(cmap="RdYlGn")

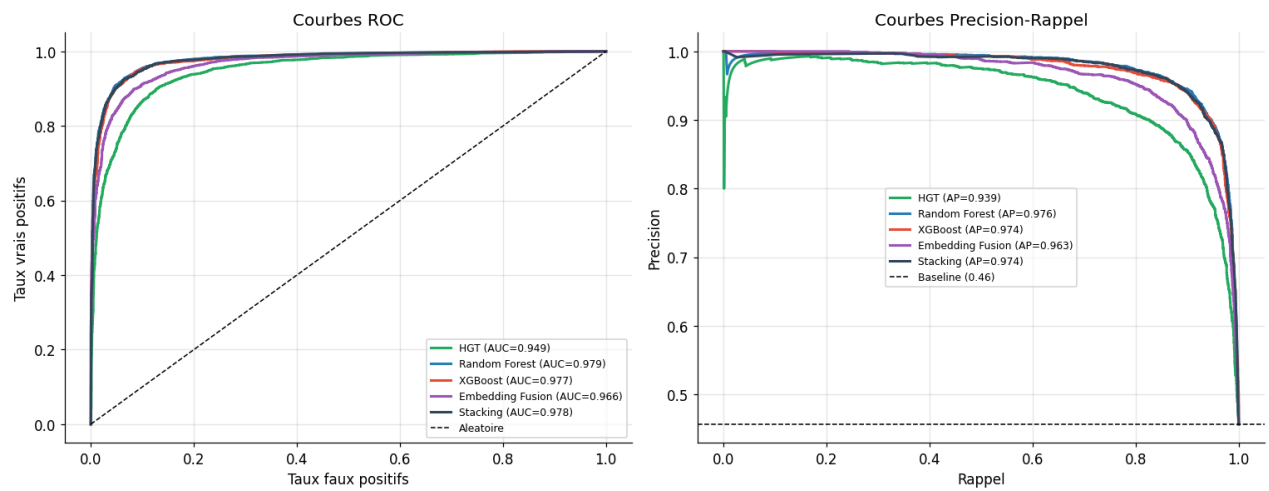
```

✓ 13.1 — Courbes ROC et Precision-Rappel

```

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5.5))
prev = y_test.mean()
for m in ORDER:
    pr = RESULTS[m]["y_proba"]; c = COLORS[m]
    fpr, tpr, _ = roc_curve(y_test, pr)
    ax1.plot(fpr, tpr, lw=2, color=c, label=f"{m} (AUC={RESULTS[m]['metrics']['roc_auc']:.3f})")
    p, r, _ = precision_recall_curve(y_test, pr)
    ax2.plot(r, p, lw=2, color=c, label=f"{m} (AP={RESULTS[m]['metrics']['avg_precision']:.3f})")
ax1.plot([0, 1], [0, 1], "k--", lw=1, label="Aleatoire")
ax2.axhline(prev, color="k", ls="--", lw=1, label=f"Baseline ({prev:.2f})")
ax1.set(xlabel="Taux faux positifs", ylabel="Taux vrais positifs", title="Courbes ROC")
ax2.set(xlabel="Rappel", ylabel="Precision", title="Courbes Precision-Rappel")
ax1.legend(fontsize=8); ax2.legend(fontsize=8); ax1.grid(alpha=.3); ax2.grid(alpha=.3)
plt.tight_layout(); savefig(fig, "fig24_roc_pr_all.png"); plt.show()

```



13.2 — Courbe de calibration (fiabilité des probabilités)

```
fig, ax = plt.subplots(figsize=(7, 6))
ax.plot([0, 1], [0, 1], "k--", label="Calibration parfaite")
for m, mk in zip(ORDER, ["o-", "s-", "^-", "v-", "D-"]):
    fr, mp = calibration_curve(y_test, RESULTS[m]["y_proba"], n_bins=10, strategy="quantile")
    ax.plot(mp, fr, mk, color=COLORS[m], lw=2, label=m)
ax.set(xlabel="Probabilite predite moyenne", ylabel="Fraction observee de positifs",
       title="Courbe de calibration")
ax.legend(fontsize=9); ax.grid(alpha=.3)
plt.tight_layout(); savefig(fig, "fig25_calibration_all.png"); plt.show()
```

13.3 — Matrices de confusion (vue d'ensemble)

```
n = len(ORDER)
fig, axes = plt.subplots(1, n, figsize=(4*n, 3.8))
if n == 1: axes = [axes]
for ax, m in zip(axes, ORDER):
    ConfusionMatrixDisplay.from_predictions(y_test, RESULTS[m]["y_pred"],
    display_labels=["0", "1"], cmap="Blues", ax=ax, colorbar=False)
    ax.set_title(m, fontsize=10)
plt.tight_layout(); savefig(fig, "fig26_confusion_all.png"); plt.show()
```

13.4 — Tableau de bord comparatif

```
metrics = ["roc_auc", "avg_precision", "recall", "precision", "f1", "balanced_accuracy"]
labels = ["ROC-AUC", "AP", "Rappel", "Precision", "F1", "Bal.Acc"]
x = np.arange(len(labels)); w = 0.8 / len(ORDER)
fig, ax = plt.subplots(figsize=(13, 6))
for i, m in enumerate(ORDER):
    vals = [RESULTS[m][metrics[k]] for k in metrics]
    ax.bar(x + i*w - 0.4 + w/2, vals, w, label=m, color=COLORS[m])
allv = [RESULTS[m][metrics[k]] for m in ORDER for k in metrics]
ax.set_xticks(x); ax.set_xticklabels(labels)
ax.set_ylim(max(0, min(allv) - 0.05), 1.0)
ax.set_ylabel("Score (test set)"); ax.set_title("Synthese des performances - cible EPA 2024 NPDWR")
ax.legend(fontsize=9, ncol=len(ORDER)); ax.grid(axis="y", alpha=.3)
plt.tight_layout(); savefig(fig, "fig27_dashboard.png"); plt.show()
```

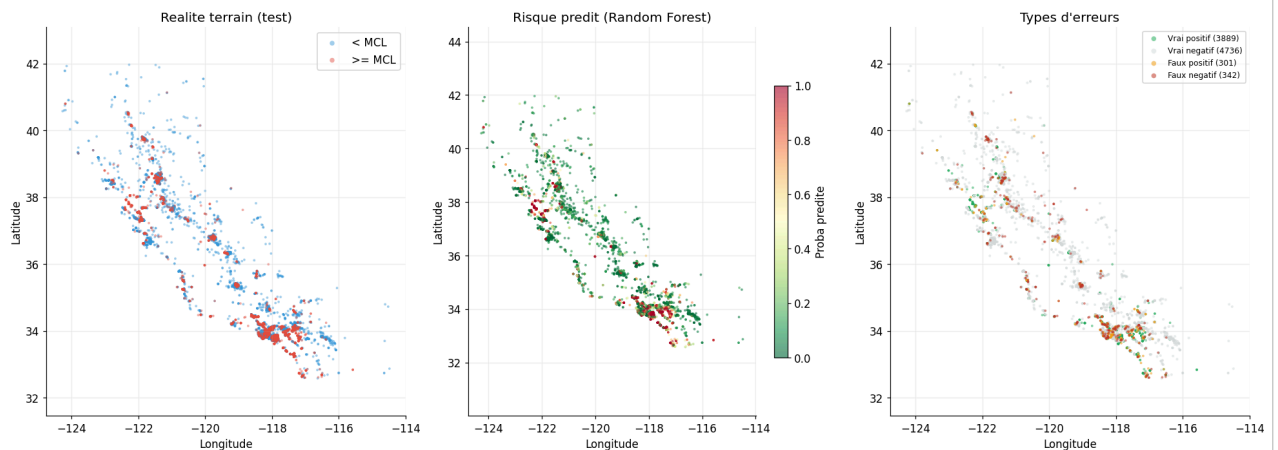
13.5 — Carte spatiale des predictions (meilleur modele)

Bien que la localisation soit volontairement laissée en feature, cette carte illustre la distribution géographique des erreurs (utile en soutenance).

```

if {"latitude", "longitude"}.issubset(df.columns):
    test_pos = X_test.index
    coords = df.loc[test_pos, ["longitude", "latitude"]].copy()
    coords["y_true"] = y_test
    coords["proba"] = RESULTS[best]["y_proba"]
    coords["y_pred"] = RESULTS[best]["y_pred"]
    fig, axes = plt.subplots(1, 3, figsize=(18, 6.5))
    for lab, c, mlab in [(0, "#3498db", "< MCL"), (1, "#e74c3c", ">= MCL")]:
        s = coords[coords.y_true == lab]
        axes[0].scatter(s.longitude, s.latitude, s=6, c=c, alpha=.45, label=mlab, edgecolors="none")
    axes[0].set_title("Realite terrain (test)"); axes[0].legend(markerscale=2)
    sc = axes[1].scatter(coords.longitude, coords.latitude, s=6, c=coords.proba,
                        cmap="RdYlGn_r", vmin=0, vmax=1, alpha=.6, edgecolors="none")
    plt.colorbar(sc, ax=axes[1], shrink=.7, label="Proba predite")
    axes[1].set_title(f"Risque predict ({best})")
    conds = [(coords.y_true==1)&(coords.y_pred==1), (coords.y_true==0)&(coords.y_pred==0),
             (coords.y_true==0)&(coords.y_pred==1), (coords.y_true==1)&(coords.y_pred==0)]
    labs = ["Vrai positif", "Vrai negatif", "Faux positif", "Faux negatif"]
    coords["err"] = np.select(conds, labs, default="?")
    pal = {"Vrai positif": "#27ae60", "Vrai negatif": "#d5dbdb", "Faux positif": "#f39c12", "Faux negatif": "#c0392b"}
    for cat, c in pal.items():
        s = coords[coords.err == cat]
        axes[2].scatter(s.longitude, s.latitude, s=6, c=c, alpha=.55, label=f"{cat} ({len(s)}", edgecolors="non
    axes[2].set_title("Types d'erreurs"); axes[2].legend(markerscale=2, fontsize=8)
    for ax in axes:
        ax.set_xlabel("Longitude"); ax.set_ylabel("Latitude"); ax.set_aspect("equal", "datalim"); ax.grid(alpha=
    plt.tight_layout(); savefig(fig, "fig28_map_predictions.png"); plt.show()

```



14. Evaluation honnête — Validation croisée spatiale par blocs (k=8)

Section activable : `RUN_SPATIAL_CV = True` dans la configuration.

Le split aléatoire 60/20/20 **surestime** la généralisation : des échantillons proches (même puits, même panache) se retrouvent à la fois en train et en test. On évalue donc la **vraie capacité hors-zone** avec une **CV spatiale par blocs** : KMeans sur (lat, lon) → **k=8** régions ; chaque région est tour à tour la *test*, le reste l'*entraînement*. On ajoute une **ablation géographique** (avec / sans **latitude**, **longitude**) pour séparer le signal mécaniste du simple proxy de position. C'est le protocole de référence pour juger si un modèle PFAS est réellement prédictif.

```

if RUN_SPATIAL_CV and {"latitude", "longitude"}.issubset(df.columns):
    from sklearn.model_selection import StratifiedKFold as _SKF
    yall = df["target"].values
    blocks = KMeans(SPATIAL_CV_K, random_state=RANDOM_STATE, n_init=10).fit_predict(
        StandardScaler().fit_transform(df[["latitude", "longitude"]].values))
    print("Tailles des blocs spatiaux :", np.bincount(blocks))
    ARMS = {"avec_geo": feature_names,
            "sans_geo": [c for c in feature_names if c not in ("latitude", "longitude")]}

    def _proc(cols, tr, te):
        nc = [c for c in cols if c in num_cols]; cc = [c for c in cols if c in cat_cols]
        ct = ColumnTransformer([

```



```

("num", Pipeline([("im", SimpleImputer(strategy="median"))], nc),
("cat", Pipeline([("im", SimpleImputer(strategy="constant", fill_value="Unknown")),
                  ("en", OrdinalEncoder(handle_unknown="use_encoded_value",
                                         unknown_value=-1, encoded_missing_value=-1))], cc)))
return ct.fit_transform(df.loc[tr, cols]), ct.transform(df.loc[te, cols])

rows = []
for arm, cols in ARMS.items():
    for f in range(SPATIAL_CV_K):
        te = blocks == f; tr = ~te
        if te.sum() < 50 or len(np.unique(yall[te])) < 2:
            continue
        Xtr, Xte = _proc(cols, tr, te)
        ytr, yte = yall[tr], yall[te]
        spw = (ytr == 0).sum() / max(1, (ytr == 1).sum())
        for name, mdl in [
            ("XGBoost", xgb.XGBClassifier(n_estimators=300, max_depth=6, learning_rate=0.1,
                                          subsample=0.8, colsample_bytree=0.8, scale_pos_weight=spw,
                                          tree_method="hist", random_state=RANDOM_STATE, eval_metric="logloss")),
            ("Random Forest", RandomForestClassifier(n_estimators=300, class_weight="balanced",
                                                    n_jobs=-1, random_state=RANDOM_STATE))]:
            mdl.fit(Xtr, ytr); p = mdl.predict_proba(Xte)[: , 1]
            rows.append(dict(model=name, arm=arm, fold=f, n_test=int(te.sum()),
                            prev=float(yte.mean()), AUC=roc_auc_score(yte, p),
                            AP=average_precision_score(yte, p)))
        print(f" [{arm}] fold {f} ok (n_test={int(te.sum())}, prev={yte.mean():.2f})")
cv_sp = pd.DataFrame(rows)
cv_sp.to_csv(RES_DIR / "cv_spatiale.csv", index=False)
summary = cv_sp.groupby(["model", "arm"]).agg(
    AUC_moy=("AUC", "mean"), AUC_std=("AUC", "std"), AP_moy=("AP", "mean")).round(4)
print("\n===== CV SPATIALE (AUC honnete hors-zone) =====")
print(summary.to_string())
else:
    print("CV spatiale desactivee (RUN_SPATIAL_CV=False).")

```

CV spatiale desactivee (RUN_SPATIAL_CV=False).

```

# Contraste CV aleatoire (optimiste) vs CV spatiale (honnete)
if RUN_SPATIAL_CV and "cv_sp" in dir() and len(cv_sp):
    from sklearn.model_selection import StratifiedKFold as _SKF
    Xall = df[feature_names]; yall = df["target"].values
    rnd = []
    for tri, tei in _SKF(5, shuffle=True, random_state=RANDOM_STATE).split(Xall, yall):
        ct = ColumnTransformer([
            ("num", SimpleImputer(strategy="median"), num_cols),
            ("cat", Pipeline([("im", SimpleImputer(strategy="constant", fill_value="Unknown")),
                              ("en", OrdinalEncoder(handle_unknown="use_encoded_value",
                                                     unknown_value=-1, encoded_missing_value=-1))], cat_cols))]
        Xt = ct.fit_transform(Xall.iloc[tri]); Xv = ct.transform(Xall.iloc[te])
        m = xgb.XGBClassifier(n_estimators=300, max_depth=6, learning_rate=0.1, subsample=0.8,
                              colsample_bytree=0.8, tree_method="hist", random_state=RANDOM_STATE,
                              eval_metric="logloss").fit(Xt, yall[tri])
        rnd.append(roc_auc_score(yall[te], m.predict_proba(Xv)[: , 1]))
    auc_rand = float(np.mean(rnd))
    auc_spat = float(cv_sp[(cv_sp.model == "XGBoost") & (cv_sp.arm == "avec_geo")].AUC.mean())
    print(f"AUC CV aleatoire (optimiste) : {auc_rand:.3f}")
    print(f"AUC CV spatiale (honnete) : {auc_spat:.3f}")
    print(f"Penalite spatiale Delta : {auc_rand - auc_spat:.3f}")

    g = cv_sp.groupby(["model", "arm"]).AUC.agg(["mean", "std"]).reset_index()
    fig, ax = plt.subplots(figsize=(8, 4.5))
    labs = [f"{r.model}\n{r.arm}" for _, r in g.iterrows()]
    ax.bar(range(len(g)), g["mean"], yerr=g["std"], capsize=4,
           color=[ "#2980b9", "#3498db", "#e74c3c", "#c0392b" ][:len(g)], edgecolor="black")
    ax.axhline(0.5, ls="--", c="grey", label="Aleatoire (0.5)")
    ax.axhline(auc_rand, ls=":", c="red", label=f"CV aleatoire ({auc_rand:.2f})")
    ax.set_xticks(range(len(g))); ax.set_xticklabels(labs, fontsize=8)
    ax.set_ylabel("ROC-AUC (CV spatiale)"); ax.set_ylim(0.4, 1.0); ax.legend(fontsize=8)
    ax.set_title("AUC honnete hors-zone vs optimisme du split aleatoire")
    plt.tight_layout(); savefig(fig, "fig29_cv_spatiale.png"); plt.show()

```

15. Importance des features & concordance des modeles

On compare les **classements de features** des modeles entraines sur la **memme matrice** (`proc_cols`) : Random Forest (importance par impurete MDI), XGBoost (gain), et HGT (norme L1 des poids d'entree). La **concordance** mesure a quel point les modeles s'accordent sur les variables importantes — un fort accord renforce la **confiance** dans l'interpretation.

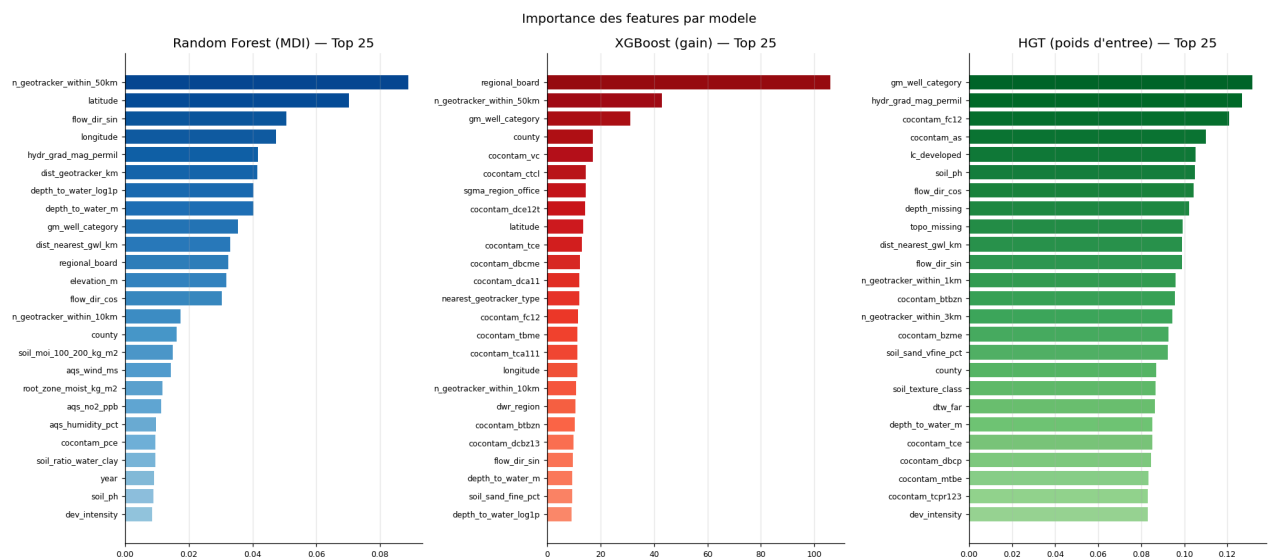
15.1 — Importance par modele (Top 25)

```
# — Random Forest : importance MDI —————
fi_rf = (pd.DataFrame({"feature": proc_cols, "imp_rf": rf.feature_importances_})
        .sort_values("imp_rf", ascending=False).reset_index(drop=True))
fi_rf["rank_rf"] = fi_rf.index + 1

# — XGBoost : gain (mappe sur l'ordre des colonnes fI -> proc_cols) ———
_gain = xgb_model.get_booster().get_score(importance_type="gain")
fi_xgb = (pd.DataFrame({"feature": proc_cols,
                        "imp_xgb": [_gain.get(f"f{i}", 0.0) for i in range(len(proc_cols))]}))
        .sort_values("imp_xgb", ascending=False).reset_index(drop=True))
fi_xgb["rank_xgb"] = fi_xgb.index + 1

# — HGT : norme L1 des poids d'entree du noeud 'sample' —————
fi_hgt = None
if TORCH_GEOMETRIC_AVAILABLE:
    with torch.no_grad():
        _w = hgt_model.lin_dict["sample"].weight.detach().cpu().numpy()
        fi_hgt = (pd.DataFrame({"feature": proc_cols, "imp_hgt": np.abs(_w).mean(0)}))
        .sort_values("imp_hgt", ascending=False).reset_index(drop=True))
        fi_hgt["rank_hgt"] = fi_hgt.index + 1

TOPN = 25
panels = [("Random Forest (MDI)", fi_rf, "imp_rf", "Blues"),
          ("XGBoost (gain)", fi_xgb, "imp_xgb", "Reds")]
if fi_hgt is not None:
    panels.append(("HGT (poids d'entree)", fi_hgt, "imp_hgt", "Greens"))
fig, axes = plt.subplots(1, len(panels), figsize=(6*len(panels), 8))
if len(panels) == 1: axes = [axes]
for ax, (title, fi, col, cmap) in zip(axes, panels):
    top = fi.head(TOPN)
    ax.barh(top["feature"][::-1], top[col][::-1],
            color=plt.get_cmap(cmap)(np.linspace(0.4, 0.9, TOPN)))
    ax.set_title(f"{title} — Top {TOPN}"); ax.grid(axis="x", alpha=.3); ax.tick_params(labelsize=8)
plt.suptitle("Importance des features par modele", fontsize=13)
plt.tight_layout(); savefig(fig, "fig40_importance_par_modele.png"); plt.show()
```



15.2 — Concordance des modeles (rang des 30 premieres features)

Pour chaque paire de modeles : nombre de features communes dans le **Top 30**, et **correlation de rang de Spearman** sur l'ensemble des features (1 = classements identiques).

```

from scipy.stats import spearmanr

# table fusionnee des rangs
merged = fi_rf[["feature", "rank_rf"]].merge(fi_xgb[["feature", "rank_xgb"]], on="feature")
if fi_hgt is not None:
    merged = merged.merge(fi_hgt[["feature", "rank_hgt"]], on="feature")
rank_cols = [c for c in ["rank_rf", "rank_xgb", "rank_hgt"] if c in merged.columns]
merged["rank_moyen"] = merged[rank_cols].mean(axis=1)

# overlap Top 30 + Spearman par paire
TOP = 30
tops = {"RF": set(fi_rf.head(TOP)["feature"]), "XGB": set(fi_xgb.head(TOP)["feature"])}
if fi_hgt is not None:
    tops["HGT"] = set(fi_hgt.head(TOP)["feature"])
pairs = [{"RF", "XGB"}] + [{"RF", "HGT"}, {"XGB", "HGT"}] if fi_hgt is not None else []
print(f"Concordance (Top {TOP}) : ")
for a, b in pairs:
    inter = len(tops[a] & tops[b])
    ca, cb = {"RF": "rank_rf", "XGB": "rank_xgb", "HGT": "rank_hgt"}[a], {"RF": "rank_rf", "XGB": "rank_xgb", "HGT": "rank_hgt"}[b]
    rho = spearmanr(merged[ca], merged[cb]).correlation
    print(f" {a:3s} vs {b:3s} : {inter:2d}/{TOP} features communes | Spearman rho = {rho:+.3f}")

if fi_hgt is not None:
    common_all = tops["RF"] & tops["XGB"] & tops["HGT"]
    print(f"\n Communes aux 3 modeles (Top {TOP}) : {len(common_all)}")

# table des features communes RF n XGB, classees par rang moyen
common_rx = tops["RF"] & tops["XGB"]
tbl = (merged[merged["feature"].isin(common_rx)]
       .sort_values("rank_moyen")["feature"] + rank_cols + ["rank_moyen"])
       .reset_index(drop=True))
tbl.to_csv(RES_DIR / "concordance_importance.csv", index=False)
print(f"\nFeatures communes RF n XGB (Top {TOP}), classees par rang moyen :")
print(tbl.head(20).to_string(index=False))

```

```

Concordance (Top 30) :
RF vs XGB : 13/30 features communes | Spearman rho = +0.066
RF vs HGT : 14/30 features communes | Spearman rho = +0.212
XGB vs HGT : 13/30 features communes | Spearman rho = +0.350

```

```
Communes aux 3 modeles (Top 30) : 8
```

```

Features communes RF n XGB (Top 30), classees par rang moyen :
   feature  rank_rf  rank_xgb  rank_hgt  rank_moyen
gm_well_category      9         3         1    4.333333
   flow_dir_sin       3        22        11   12.000000
   county            15         4        17   12.000000
   regional_board     11         1        27   13.000000
   depth_to_water_m     8        23        20   17.000000
   depth_to_water_loglp  7        25        30   20.666667
   cocontam_dcall      28        12        26   22.000000
n_geotracker_within_10km 14        18        41   24.333333
   soil_texture_class   30        28        18   25.333333
   longitude           4        17        63   28.000000
n_geotracker_within_50km  1         2        87   30.000000
   cocontam_pce        21        26        44   30.333333
   latitude           2         9        93   34.666667

```

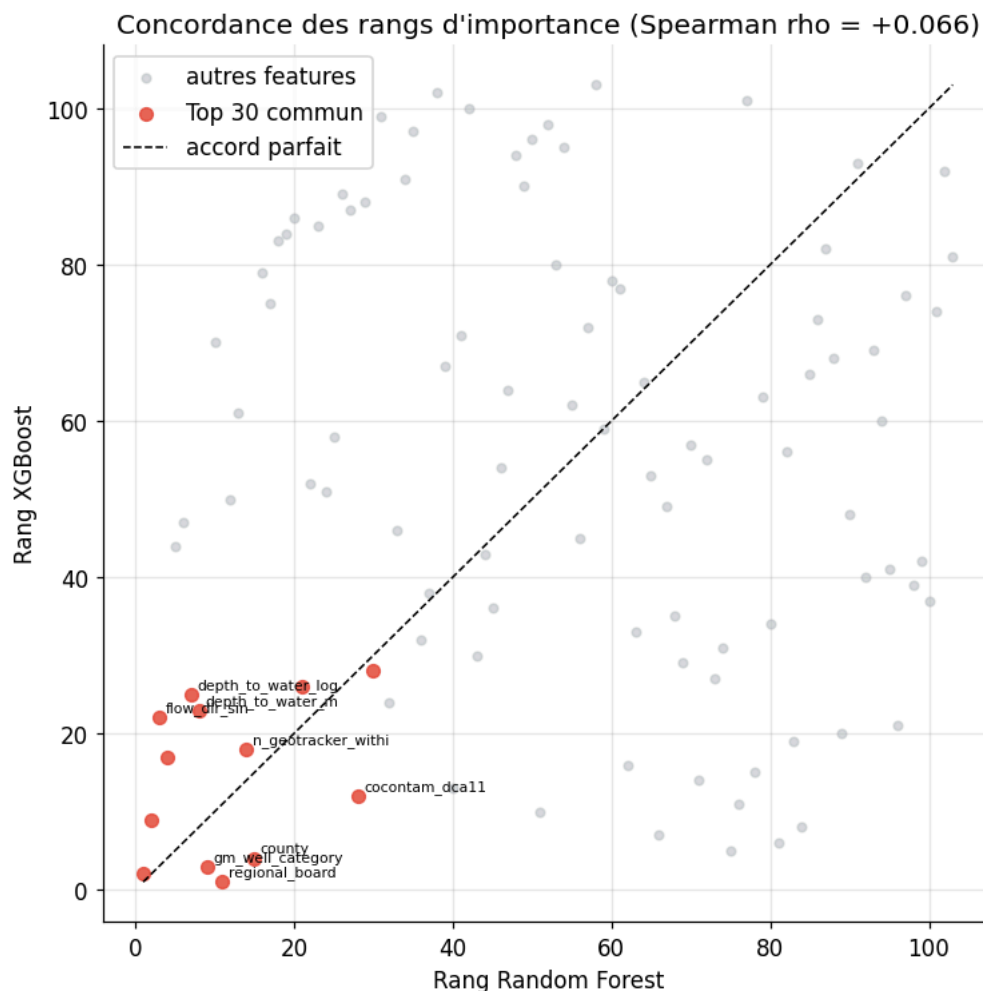
15.3 — Nuage des rangs (RF vs XGBoost)

Chaque point = une feature ; plus elle est proche de la diagonale, plus les deux modeles s'accordent sur son importance. Les features du **Top 30 commun** sont mises en evidence.

```

fig, ax = plt.subplots(figsize=(7, 7))
on_diag = merged["feature"].isin(common_rx)
ax.scatter(merged.loc[~on_diag, "rank_rf"], merged.loc[~on_diag, "rank_xgb"],
           s=18, c="#bdc3c7", alpha=.6, label="autres features")
ax.scatter(merged.loc[on_diag, "rank_rf"], merged.loc[on_diag, "rank_xgb"],
           s=40, c="#e74c3c", alpha=.85, label=f"Top {TOP} commun")
lim = len(proc_cols)
ax.plot([1, lim], [1, lim], "k--", lw=1, label="accord parfait")
# annoter les 8 features les mieux classees en moyenne
for _, r in tbl.head(8).iterrows():
    ax.annotate(r["feature"][:18], (r["rank_rf"], r["rank_xgb"]), fontsize=7,
               xytext=(3, 3), textcoords="offset points")
ax.set(xlabel="Rang Random Forest", ylabel="Rang XGBoost",
       title=f"Concordance des rangs d'importance (Spearman rho = {spearmanr(merged.rank_rf, merged.rank_xgb).co
ax.legend(); ax.grid(alpha=.3)
plt.tight_layout(); savefig(fig, "fig41_concordance_rangs.png"); plt.show()

```



16. Exploration pedagogique du HGT — « sous le capot »

Cette section ouvre la **boîte noire** du graphe heterogene et du message-passing, pour la soutenance : structure du graphe, contenu exact de chaque **noeud** et **arete**, **voisinage** d'un puits, **calcul explicite d'une couche HGT** sur ce voisinage, et projection 2D des embeddings appris. Tout est calcule sur le **graphe de test** deja construit.

16.1 — Vue d'ensemble du graphe (noeuds & aretes)

```
if TORCH_GEOMETRIC_AVAILABLE:
    g = test_data
    nodes_tbl = pd.DataFrame([
        {"type de noeud": nt, "nb noeuds": g[nt].num_nodes,
         "dim features": (g[nt].x.shape[1] if "x" in g[nt] else 0)}
        for nt in g.node_types])
    edges_tbl = pd.DataFrame([
        {"relation (src -[rel]-> dst)": f"{a} -[{b}]-> {c}", "nb aretes": g[a, b, c].edge_index.shape[1]}
        for a, b, c in g.edge_types])
    print("NOEUDS\n", nodes_tbl.to_string(index=False))
    print("\nARETES\n", edges_tbl.to_string(index=False))
    print(f"\nTotal : {sum(g[nt].num_nodes for nt in g.node_types):,} noeuds | "
          f"{sum(g[a,b,c].edge_index.shape[1] for a,b,c in g.edge_types):,} aretes")
```

NOEUDS

	type de noeud	nb noeuds	dim features
	sample	9268	103
	Facility_Proximity	9268	5
	Water_Quality_cocontam	9268	39
	Soil_SSURGO	9268	16
	Air_AQS	9268	8
	Environmental_GLDAS_hydro	9268	11
	Hydrogeology	9268	12
	Temporal	9268	3
	geo_cluster	20	2

ARETES

	relation (src -[rel]-> dst)	nb aretes
	sample -[has_Facility_Proximity]-> Facility_Proximity	9268
	Facility_Proximity -[in_Facility_Proximity]-> sample	9268

```

sample -[has_Water_Quality_cocontam]-> Water_Quality_cocontam 9268
Water_Quality_cocontam -[in_Water_Quality_cocontam]-> sample 9268
sample -[has_Soil_SSURGO]-> Soil_SSURGO 9268
Soil_SSURGO -[in_Soil_SSURGO]-> sample 9268
sample -[has_Air_AQS]-> Air_AQS 9268
Air_AQS -[in_Air_AQS]-> sample 9268
sample -[has_Environmental_GLDAS_hydro]-> Environmental_GLDAS_hydro 9268
Environmental_GLDAS_hydro -[in_Environmental_GLDAS_hydro]-> sample 9268
sample -[has_Hydrogeology]-> Hydrogeology 9268
Hydrogeology -[in_Hydrogeology]-> sample 9268
sample -[has_Temporal]-> Temporal 9268
Temporal -[in_Temporal]-> sample 9268
sample -[located_in]-> geo_cluster 9268
geo_cluster -[contains]-> sample 9268

```

Total : 74,164 noeuds | 148,288 aretes

✓ 16.2 — Constitution de chaque noeud (features)

- **sample** : un puits = le vecteur complet des features standardisees (sans localisation si applicable).
- **noeuds de groupe** (**Water_Quality_cocontam**, **Soil_SSURGO**, ...): sous-ensemble thematique des features.
- **geo_cluster** : centre du cluster KMeans = (latitude, longitude) standardisees.

```

if TORCH_GEOMETRIC_AVAILABLE:
    def describe_node(nt, names=None, k=8):
        x = g[nt].x
        print(f"\n[{nt}] shape={tuple(x.shape)}")
        if names is not None:
            ex = x[0].cpu().numpy()
            for nm, v in list(zip(names, ex))[:k]:
                print(f"    {nm:28s} = {v:+.3f}")
            if len(names) > k:
                print(f"    ... ({len(names)-k} autres features)")
        else:
            print("    exemple (noeud 0) :", np.round(x[0].cpu().numpy()[:6], 3))

# noeud sample : toutes les features
describe_node("sample", proc_cols)
# noeuds de groupe : features de chaque groupe
for grp, feats in list(GRAPH_GROUPS.items()):
    if grp in g.node_types:
        describe_node(grp, [f for f in feats if f in proc_cols])
# geo_cluster : centres lat/lon
if "geo_cluster" in g.node_types:
    describe_node("geo_cluster", ["centre_lat (z)", "centre_lon (z)"])

```

```

[sample] shape=(9268, 103)
latitude           = -0.776
longitude          = +0.951
dist_geotracker_km = -0.513
n_geotracker_within_1km = -0.332
n_geotracker_within_3km = +0.359
n_geotracker_within_10km = -0.082
n_geotracker_within_50km = +0.181
cocontam_btbszs    = -0.217
... (+95 autres features)

```

```

[Facility_Proximity] shape=(9268, 5)
dist_geotracker_km    = -0.513
n_geotracker_within_1km = -0.332
n_geotracker_within_3km = +0.359
n_geotracker_within_10km = -0.082
n_geotracker_within_50km = +0.181

```

```

[Water_Quality_cocontam] shape=(9268, 39)
cocontam_btbszs          = -0.217
cocontam_pbzn            = -0.107
cocontam_edb             = +0.110
cocontam_dbcp            = +0.768
cocontam_tcpr123         = -0.006
cocontam_tce             = -0.118
cocontam_pce             = -0.138
cocontam_mtbe            = +0.065
... (+31 autres features)

```

```

[Soil_SSURGO] shape=(9268, 16)
soil_sand_pct            = +0.951
soil_clay_pct            = -1.184
soil_silt_pct            = -0.616
soil_om_pct             = -0.209
soil_ph                 = -0.501
soil_ksat_um_s          = +1.021
soil_awc_cm_cm          = -1.147

```